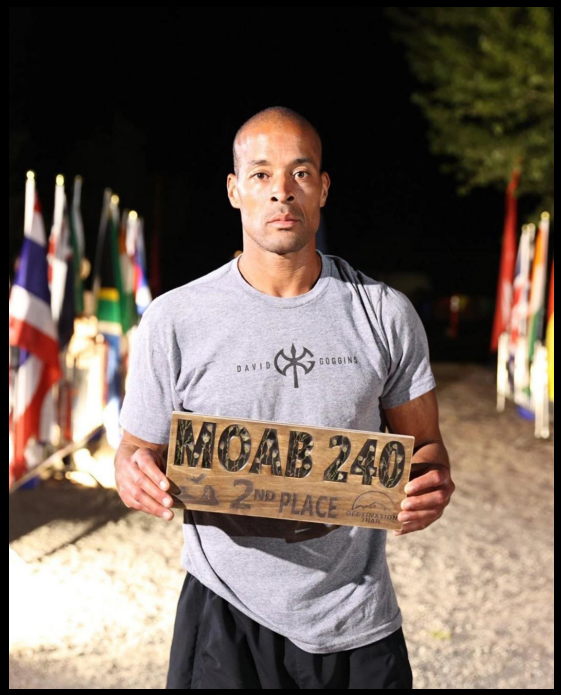
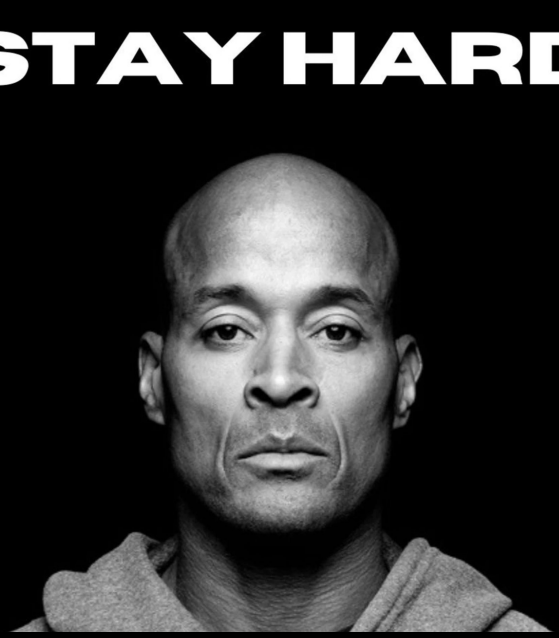
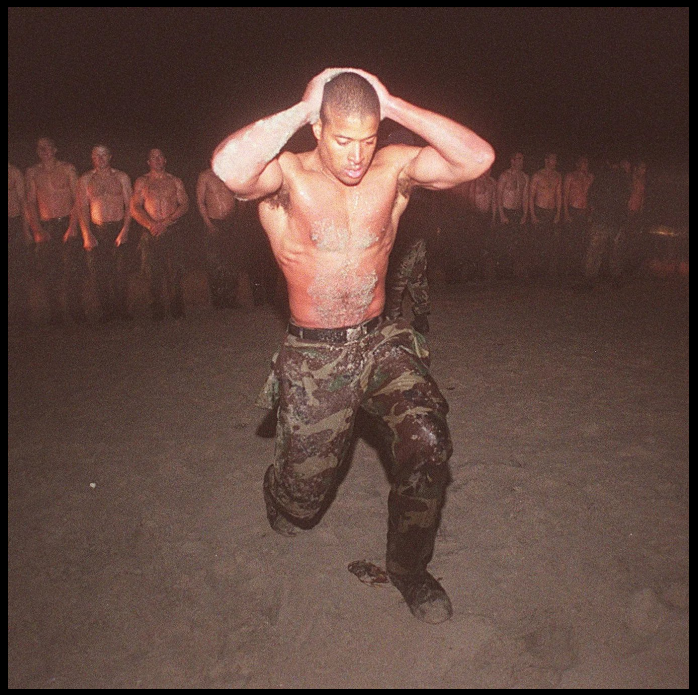
STAY HARD







STAY FOCUSED. STAY FUCKING HARD.
THEY DON'T KNOW US, SON.
WHO'S GONNA CARRY THE BOATS AND THE LOGS?
WE ARE, MOTHERFUCKERS. WE ARE.

Содержание

1 Шаблоны 2
1.1 Шаблон . . . 2
1.2 Быстрый шаблон (без solve) . . . 2
1.3 Аллокатор . . . 2
1.4 Компиляция . . . 3
1.5 Стресс-тест . . . 3

2 Структуры данных 3
2.1 Фенвик . . . 3
2.2 Sparse table . . . 3
2.3 Disjoint sparse table . . . 3
2.4 Дерево отрезков (ленивое) . . . 3
2.5 Дерево отрезков снизу . . . 4
2.6 CHM с откатами . . . 4
2.7 Неявное декартово дерево . . . 4
2.8 Персистентное декартово дерево . . . 5
2.9 Link-Cut Tree . . . 5
2.10 Алгоритм Мо . . . 6
2.11 Динамический KXT . . . 6
2.12 Li Chao . . . 6

3 Графы 7
3.1 Диниц, Гомори–Ху, Штор–Вагнер . . . 7
3.2 Mincost maxflow . . . 7
3.3 Паросочетания: Хопкрофт–Карп, блоссом, венгерка . . . 8
3.4 KCC и 2-SAT . . . 9
3.5 Мосты, точки сочленения, BCC . . . 9
3.6 HLD и центроидная декомпозиция . . . 9
3.7 Оффлайн динамическая связность . . . 10
3.8 Пути, Эйлеров путь, направленный остов . . . 10

4 Математика 11
4.1 Теория чисел: gcd, КТО, решето . . . 11
4.2 Миллер–Рабин, Ро-Поллард . . . 11
4.3 sqrt, первообразный корень, лог, floor-sum . . . 12
4.4 Суммы по делителям, Мёбиус, НОД/НОК-свёртки . . . 12
4.5 Гаусс, определитель, матричная степень . . . 12
4.6 Берлекэмп–Мэсси, n-й член, Лагранж . . . 13
4.7 Комбинаторика, Лукас, pi(n) . . . 13
4.8 Симплекс . . . 14

5 Преобразования и оптимизации 14
5.1 NTT и свёртка по модулю . . . 14
5.2 FFT (double) . . . 14
5.3 Многочлены: inv, log, exp, Стирлинг-строки . . . 15
5.4 Уолш–Адамар, SOS . . . 15
5.5 D&C, Кнут, HBП . . . 15

6 Строки 16
6.1 Z, префикс, Манакер, суфмас, хеш . . . 16
6.2 Автомат, Ахо-Корасик, эертри . . . 16

7 Геометрия 17
7.1 База: точка, прямая, окружность . . . 17
7.2 Оболочка, полигоны, ПП, калиперы, круг и полигон . . . 18

8 Разное 19
8.1 pbds, хеш, быстрый ввод, int128 . . . 19

9 Факты 21
9.1 Обозначения (читать первым) . . . 21
9.2 Как решать задачи на подсчёт (шпаргалка) . . . 21
9.3 Биномиальные коэффициенты . . . 21
9.4 Разложение на слагаемые (шарики и перегородки) . . . 21
9.5 Каталан и скобочные последовательности . . . 21
9.6 Числа Стирлинга, Белла, Эйлера . . . 21
9.7 Перестановки и симметрии . . . 22
9.8 Разбиения числа . . . 22
9.9 Деревья, остовы, эйлеровы циклы . . . 22
9.10 Производящие функции (помеченные объекты, EGF) . . . 22
9.11 Суммы, ряды, рекуррентности . . . 22
9.12 Модульная арифметика . . . 22
9.13 Мёбиус и суммы по делителям . . . 22
9.14 Суммы квадратов, тройки, решётка . . . 23
9.15 Фибоначчи . . . 23
9.16 Асимптотики . . . 23
9.17 Таблицы . . . 23
9.18 Геометрия на плоскости . . . 23
9.19 Теоремы о графах . . . 23
9.20 Игры . . . 24
9.21 Вероятность . . . 24
9.22 Приёмы и грабли . . . 24
9.23 Ловкие приёмы и когда их применять . . . 24

1. Шаблоны

Шаблон

```
#include <bits/stdc++.h>

// #pragma GCC optimize("O3,unroll-loops")
// #pragma GCC target("avx2")

#define all(x) x.begin(), x.end()
#define rall(x) x.rbegin(), x.rend()
// #define int long long

using namespace std;
using ll = long long;
using ull = unsigned long long;
using ld = long double;
using pii = pair<int, int>;
using pll = pair<long long, long long>;
template<typename T1, typename T2>
using pr = pair<T1, T2>;
using vi = vector<int>;
using vb = vector<bool>;
using vii = vector<pair<int, int>>;
using vll = vector<long long>;
template<typename T>
using ve = vector<T>;

const long long MOD = 1e9 + 7;
const int INF = 1e9 + 1;
const long long INFLL = 1e18 + 1;

inline void fchmin(int &a, int b) { a = min(a, b); }
inline void fchmax(int &a, int b) { a = max(a, b); }

template<typename T>
inline bool chmin(T &a, T b) {
  if (a > b) {
    a = b;
    return true;
  }
  return false;
}

template<typename T>
inline bool chmax(T &a, T b) {
  if (a < b) {
    a = b;
    return true;
  }
  return false;
}

template <typename... Args>
void dbg(const string& s, const Args&... args) {
  #ifdef DEBUG
  cout << s << ": ";
  ((cout << args << ' '), ...);
  cout << '\n';
  #endif
}

int reps = 1;

void solve() {

}

signed main() {
```

```
  ios::sync_with_stdio(0);
  cin.tie(0);

  #ifdef LOCAL
  freopen("input.txt", "r", stdin);
  freopen("output.txt", "w", stdout);
  #endif
  cin >> reps;

  for (int i = 1; i <= reps; i++) {
    #ifdef LOCAL
    cout << "Test #" << i << " ================\n";
    #endif
    solve();
    #ifdef LOCAL
    cout << "End test ===============\n\n";
    #endif
  }
  #ifdef LOCAL
  cout << "Runtime is: " << clock() * 1.0 / CLOCKS_PER_SEC <<
     endl;
  #endif
}
```

Быстрый шаблон (без solve)

```
#include <bits/stdc++.h>

// #pragma GCC optimize("O3,unroll-loops")
// #pragma GCC target("avx2")

#define all(x) x.begin(), x.end()
#define rall(x) x.rbegin(), x.rend()
// #define int long long

using namespace std;
using ll = long long;
using ull = unsigned long long;
using ld = long double;
using pii = pair<int, int>;
template<typename T1, typename T2>
using pr = pair<T1, T2>;
using vi = vector<int>;
using vb = vector<bool>;
using vii = vector<pair<int, int>>;
using vll = vector<long long>;
template<typename T>
using ve = vector<T>;

const long long MOD = 1e9 + 7;
const int INF = 1e9 + 1;
const long long INFLL = 1e18 + 1;

signed main() {
  ios::sync_with_stdio(0);
  cin.tie(0);

  #ifdef LOCAL
  freopen("input.txt", "r", stdin);
  freopen("output.txt", "w", stdout);
  #endif

  return 0;
}
```

Аллокатор

```
const int MAX_MEM = 1e8;

int mpos = 0;
alignas(long long) char mem[MAX_MEM];

inline void * operator new(size_t n) {
  if (n & 7) n += 8 - (n & 7);
  char *res = mem + mpos;
  mpos += n;
  return (void *)res;
}

void operator delete(void *) {}
```

Компиляция

```
#!/bin/bash

a=$1
/opt/homebrew/bin/g++-15 -D_Alignof=__alignof__ -std=c++23 -
    Wall -Wextra -Wshadow -g -DLOCAL -D_GLIBCXX_DEBUG -O2 "$a
    .cpp" -o "$a"
```

Стресс-тест

```
#!/bin/bash
# Стресс-тест: ./stress.sh sol brute gen   (исходники sol.cpp
    brute.cpp gen.cpp, gen принимает seed в argv[1])
set -e
for f in "$@"; do g++ -std=c++20 -O2 -Wall -Wextra -Wshadow -
    D_GLIBCXX_DEBUG -fsanitize=address,undefined "$f.cpp" -o
    "$f"; done
for ((i = 1; ; i++)); do
  ./$3 $i > in
  ./$1 < in > out1
  ./$2 < in > out2
  diff -w out1 out2 > /dev/null || { echo "WA на тесте $i";
    break; }
  echo "ok $i"
done
```

STAY HARD.

2. Структуры данных

Фенвик

```
// Фенвик на [0, n). get(l, r) -- сумма на [l, r). lower_bound
    -- для неотрицательных
// значений: сколько первых элементов имеют суммарно <= pos.
template<typename T>
struct FenwickTree {
  int maxn;
  ve<T> ft;

  FenwickTree() {}
  FenwickTree(int n) : maxn(n), ft(n) {}
  FenwickTree(const ve<T> &s) : maxn(s.size()), ft(s) {
    for (int i = 0; i < maxn; i++) {
      int j = i | (i + 1);
      if (j < maxn) ft[j] += ft[i];
    }
  }

  void update(int i, T x) {
    for (; i < maxn; i |= (i + 1)) ft[i] += x;
  }

  T get(int r) {  // сумма на [0, r]
    T res = 0;
    for (r = min(r, maxn - 1); r >= 0; r = (r & (r + 1)) - 1)
     res += ft[r];
    return res;
  }

  T get(int ql, int qr) {
    return get(qr - 1) - get(ql - 1);
  }

  int lower_bound(T pos) {
    int k = 0;
    for (int l = 29; l >= 0; l--) {
      if (k + (1 << l) <= maxn && ft[k + (1 << l) - 1] <= pos)
      {
        k += (1 << l);
        pos -= ft[k - 1];
      }
    }
    return k;
  }
};
```

Sparse table

```
// Разреженная таблица (min), запрос на [l, r), r > l, O(1).
struct SparseTable {
  ve<vi> spt;

  SparseTable() {}
  SparseTable(const vi &a) {
    int n = a.size(), logn = 32 - __builtin_clz(max(n, 1));
    spt.assign(logn, vi(n, INF));
    spt[0] = a;
    for (int l = 0; l + 1 < logn; l++) {
      for (int i = 0; i + (2 << l) <= n; i++) {
        spt[l + 1][i] = min(spt[l][i], spt[l][i + (1 << l)]);
      }
    }
  }

  int get(int l, int r) {
    int k = 31 - __builtin_clz(r - l);
    return min(spt[k][l], spt[k][r - (1 << k)]);
  }
};
```

Disjoint sparse table

```
// Disjoint Sparse Table: запрос на [l, r) за O(1) для ЛЮБОЙ а
    ссоциативной операции
// (не только идемпотентной). Размер дополняется до степени дв
    ойки.
struct DST {
  struct Node {
    int x;
    Node() : x(INF) {}
    Node(int _x) : x(_x) {}
  };

  Node merge(const Node &l, const Node &r) {  // заменить на с
     вою операцию
    return (l.x <= r.x) ? l : r;
  }

  int n, logn;
  ve<ve<Node>> dst;
```

```
  DST() {}
  DST(const vi &a) {
    logn = 1;
    while ((1 << logn) < (int)a.size()) logn++;
    n = 1 << logn;
    dst.assign(logn, ve<Node>(n));
    for (int d = 0; d < logn; d++) {
      int len = 1 << d;
      for (int mid = len; mid < n + len; mid += 2 * len) {
        // dst[d][i], i < mid -- сумма [i, mid); i >= mid -- с
     умма [mid, i]
        if (mid - 1 < (int)a.size()) dst[d][mid - 1] = Node(a[
     mid - 1]);
        for (int i = mid - 2; i >= mid - len; i--) {
          dst[d][i] = merge(Node(i < (int)a.size() ? a[i] :
     INF), dst[d][i + 1]);
        }
        if (mid < n) dst[d][mid] = Node(mid < (int)a.size() ?
     a[mid] : INF);
        for (int i = mid + 1; i < min(n, mid + len); i++) {
          dst[d][i] = merge(dst[d][i - 1], Node(i < (int)a.
     size() ? a[i] : INF));
        }
      }
    }
  }

  Node get(int l, int r) {  // [l, r)
    r--;
    if (l == r) return dst[0][l];
    int d = 31 - __builtin_clz(l ^ r);
    return merge(dst[d][l], dst[d][r]);
  }
};
```

Дерево отрезков (ленивое)

```
// Дерево отрезков с ленивым прибавлением. Отрезки [l, r). Хра
    нит сумму и максимум.
struct SegTree {
  struct Node {
    ll sum = 0, mx = -INFLL, add = 0;
  };

  int maxn;
  ve<Node> st;

  SegTree() {}
  SegTree(const ve<ll> &a) : maxn(a.size()), st(4 * a.size() +
     4) {
    build(a, 1, 0, maxn);
  }

  void pull(int v) {
    st[v].sum = st[2 * v].sum + st[2 * v + 1].sum;
    st[v].mx = max(st[2 * v].mx, st[2 * v + 1].mx);
  }

  void apply(int v, int len, ll x) {
    st[v].sum += x * len;
    st[v].mx += x;
    st[v].add += x;
  }

  void push(int v, int l, int r) {
    if (!st[v].add) return;
    int mid = (l + r) / 2;
    apply(2 * v, mid - l, st[v].add);
    apply(2 * v + 1, r - mid, st[v].add);
    st[v].add = 0;
  }

  void build(const ve<ll> &a, int v, int l, int r) {
    if (r - l == 1) {
      st[v].sum = st[v].mx = a[l];
      return;
    }
    int mid = (l + r) / 2;
    build(a, 2 * v, l, mid);
    build(a, 2 * v + 1, mid, r);
    pull(v);
  }

  void update(int ql, int qr, ll x, int v, int l, int r) {
    if (qr <= l || r <= ql) return;
    if (ql <= l && r <= qr) {
      apply(v, r - l, x);
      return;
    }
    push(v, l, r);
    int mid = (l + r) / 2;
    update(ql, qr, x, 2 * v, l, mid);
    update(ql, qr, x, 2 * v + 1, mid, r);
    pull(v);
  }

  void update(int ql, int qr, ll x) {
    update(ql, qr, x, 1, 0, maxn);
  }

  ll getSum(int ql, int qr, int v, int l, int r) {
    if (qr <= l || r <= ql) return 0;
    if (ql <= l && r <= qr) return st[v].sum;
    push(v, l, r);
    int mid = (l + r) / 2;
    return getSum(ql, qr, 2 * v, l, mid) + getSum(ql, qr, 2 *
     v + 1, mid, r);
  }

  ll getSum(int ql, int qr) {
    return getSum(ql, qr, 1, 0, maxn);
  }

  ll getMax(int ql, int qr, int v, int l, int r) {
    if (qr <= l || r <= ql) return -INFLL;
    if (ql <= l && r <= qr) return st[v].mx;
    push(v, l, r);
    int mid = (l + r) / 2;
    return max(getMax(ql, qr, 2 * v, l, mid), getMax(ql, qr, 2
      * v + 1, mid, r));
  }

  ll getMax(int ql, int qr) {
    return getMax(ql, qr, 1, 0, maxn);
  }

  // первый i >= ql с a[i] >= x, иначе -1
  int firstGe(int ql, ll x, int v, int l, int r) {
    if (r <= ql || st[v].mx < x) return -1;
    if (r - l == 1) return l;
    push(v, l, r);
    int mid = (l + r) / 2;
    int res = firstGe(ql, x, 2 * v, l, mid);
    return (res != -1) ? res : firstGe(ql, x, 2 * v + 1, mid,
     r);
  }

  int firstGe(int ql, ll x) {
    return firstGe(ql, x, 1, 0, maxn);
  }
};
```

Дерево отрезков снизу

```
// Снизу-вверх, без рекурсии, работает и для некоммутативной о
    перации. [l, r).
// Пример: Node -- функция x -> a * x + b (mod MOD); merge(l,
    r) = сначала l, потом r.
struct SegTreeFast {
  struct Node {
    ll a, b;
    Node() : a(1), b(0) {}  // нейтральный элемент
    Node(ll _a, ll _b) : a(_a), b(_b) {}
  };

  Node merge(const Node &l, const Node &r) {
    return Node(l.a * r.a % MOD, (l.b * r.a + r.b) % MOD);
  }

  int maxn;
  ve<Node> st;

  SegTreeFast() {}
  SegTreeFast(int n) : maxn(n), st(2 * n) {}
  SegTreeFast(const ve<Node> &a) : maxn(a.size()), st(2 * a.
    size()) {
    for (int i = 0; i < maxn; i++) st[maxn + i] = a[i];
    for (int i = maxn - 1; i > 0; i--) st[i] = merge(st[2 * i
    ], st[2 * i + 1]);
  }

  void update(int i, const Node &x) {
    for (st[i += maxn] = x; i > 1; i >>= 1) {
      st[i >> 1] = merge(st[i & ~1], st[i | 1]);
    }
  }

  Node get(int l, int r) {
    Node resl, resr;
    for (l += maxn, r += maxn; l < r; l >>= 1, r >>= 1) {
      if (l & 1) resl = merge(resl, st[l++]);
      if (r & 1) resr = merge(st[--r], resr);
    }
    return merge(resl, resr);
  }
};
```

CHM с откатами

```
// CHM с откатами (без сжатия путей, поэтому O(log n) на опера
    цию).
// Для обычного CHM: убрать hist и сделать p[v] = find(p[v]).
struct DSU {
  vi p, sz;
  ve<pii> hist;  // (присоединённый корень, его новый родитель
    )

  DSU() {}
  DSU(int n) : p(n), sz(n, 1) {
    iota(all(p), 0);
  }

  int find(int v) {
    return p[v] == v ? v : find(p[v]);
```

```
  }

  bool unite(int a, int b) {
    a = find(a);
    b = find(b);
    if (a == b) return false;
    if (sz[a] < sz[b]) swap(a, b);
    p[b] = a;
    sz[a] += sz[b];
    hist.push_back({b, a});
    return true;
  }

  int snapshot() {
    return hist.size();
  }

  void rollback(int snap) {
    while ((int)hist.size() > snap) {
      auto [b, a] = hist.back();
      hist.pop_back();
      p[b] = b;
      sz[a] -= sz[b];
    }
  }
};
```

Неявное декартово дерево

```
mt19937 rng(1543);

// Неявное декартово дерево (индексы вместо указателей): встав
     ка/удаление по позиции,
// разворот и сумма на отрезке [l, r). t[0] -- null-вершина.
struct Treap {
  struct Node {
    int l = 0, r = 0, sz = 1, y = 0;
    ll x = 0, sum = 0;
    bool rev = false;
  };

  ve<Node> t;
  int root = 0;

  Treap() : t(1) {
    t[0].sz = 0;
  }

  int newNode(ll x) {
    Node v;
    v.x = v.sum = x;
    v.y = rng();
    t.push_back(v);
    return (int)t.size() - 1;
  }

  void pull(int v) {
    int l = t[v].l, r = t[v].r;
    t[v].sz = 1 + t[l].sz + t[r].sz;
    t[v].sum = t[v].x + t[l].sum + t[r].sum;
  }

  void applyRev(int v) {
    if (!v) return;
    t[v].rev ^= 1;
    swap(t[v].l, t[v].r);
  }

  void push(int v) {
    if (!t[v].rev) return;
    applyRev(t[v].l);
    applyRev(t[v].r);
    t[v].rev = false;
  }

  pii split(int v, int k) {  // первые k вершин -> левое дерев
     о
    if (!v) return {0, 0};
    push(v);
    if (t[t[v].l].sz >= k) {
      auto [a, b] = split(t[v].l, k);
      t[v].l = b;
      pull(v);
      return {a, v};
    }
    auto [a, b] = split(t[v].r, k - t[t[v].l].sz - 1);
    t[v].r = a;
    pull(v);
    return {v, b};
  }

  int merge(int a, int b) {
    if (!a || !b) return a ? a : b;
    if (t[a].y > t[b].y) {
      push(a);
      int c = merge(t[a].r, b);
      t[a].r = c;
      pull(a);
      return a;
    }
    push(b);
    int c = merge(a, t[b].l);
    t[b].l = c;
    pull(b);
    return b;
  }

  int size() { return t[root].sz; }

  void insert(int pos, ll x) {
    auto [a, b] = split(root, pos);
    root = merge(merge(a, newNode(x)), b);
  }

  void erase(int pos) {
    auto [a, b] = split(root, pos);
    auto [c, d] = split(b, 1);
    root = merge(a, d);
  }

  void reverse(int l, int r) {
    auto [a, b] = split(root, l);
    auto [c, d] = split(b, r - l);
    applyRev(c);
    root = merge(a, merge(c, d));
  }

  ll sum(int l, int r) {
    auto [a, b] = split(root, l);
    auto [c, d] = split(b, r - l);
    ll res = t[c].sum;
    root = merge(a, merge(c, d));
    return res;
  }

  ll get(int pos) {
    return sum(pos, pos + 1);
  }

  void dump(int v, ve<ll> &out) {  // содержимое по порядку
    if (!v) return;
    push(v);
    dump(t[v].l, out);
    out.push_back(t[v].x);
    dump(t[v].r, out);
  }
};
```

Персистентное декартово дерево

```
mt19937 prng(7);

// Персистентное неявное декартово дерево (без ленивых операци
     й). Версия = корень (int).
// Каждая операция возвращает новый корень, старые версии оста
     ются валидными.
// Память: O(log n) новых вершин на операцию.
struct PersTreap {
  struct Node {
    int l = 0, r = 0, sz = 1, y = 0;
    ll x = 0, sum = 0;
  };

  ve<Node> t;

  PersTreap() : t(1) {
    t[0].sz = 0;
  }

  int newNode(ll x) {
    Node v;
    v.x = v.sum = x;
    v.y = prng();
    t.push_back(v);
    return (int)t.size() - 1;
  }

  int copy(int v) {
    Node c = t[v];
    t.push_back(c);
    return (int)t.size() - 1;
  }

  void pull(int v) {
    int l = t[v].l, r = t[v].r;
    t[v].sz = 1 + t[l].sz + t[r].sz;
    t[v].sum = t[v].x + t[l].sum + t[r].sum;
  }

  pii split(int v, int k) {
    if (!v) return {0, 0};
    int c = copy(v);
    if (t[t[v].l].sz >= k) {
      auto [a, b] = split(t[v].l, k);
      t[c].l = b;
      pull(c);
      return {a, c};
    }
    auto [a, b] = split(t[v].r, k - t[t[v].l].sz - 1);
    t[c].r = a;
    pull(c);
    return {c, b};
  }
```

```
  int merge(int a, int b) {
    if (!a || !b) return a ? a : b;
    int c;
    if (t[a].y > t[b].y) {
      c = copy(a);
      int r = merge(t[a].r, b);
      t[c].r = r;
    } else {
      c = copy(b);
      int l = merge(a, t[b].l);
      t[c].l = l;
    }
    pull(c);
    return c;
  }

  int build(const ve<ll> &a) {
    int root = 0;
    for (ll x : a) root = merge(root, newNode(x));
    return root;
  }

  int insert(int root, int pos, ll x) {
    auto [a, b] = split(root, pos);
    return merge(merge(a, newNode(x)), b);
  }

  int erase(int root, int pos) {
    auto [a, b] = split(root, pos);
    auto [c, d] = split(b, 1);
    return merge(a, d);
  }

  ll sum(int root, int l, int r) {  // [l, r)
    auto [a, b] = split(root, l);
    auto [c, d] = split(b, r - l);
    return t[c].sum;
  }

  int size(int root) { return t[root].sz; }
};
```

Link-Cut Tree

```
// Link-Cut Tree: вершины 1..n. Лес с корнями, makeRoot/link/
    cut/connected, сумма на пути.
struct LinkCut {
  struct Node {
    int ch[2] = {0, 0}, p = 0;
    bool rev = false;
    ll val = 0, sum = 0;
  };

  ve<Node> t;

  LinkCut(int n) : t(n + 1) {}

  bool isRoot(int x) {
    int p = t[x].p;
    return !p || (t[p].ch[0] != x && t[p].ch[1] != x);
  }

  void pull(int x) {
    t[x].sum = t[x].val + t[t[x].ch[0]].sum + t[t[x].ch[1]].
     sum;
  }

  void flip(int x) {
    if (!x) return;
    swap(t[x].ch[0], t[x].ch[1]);
    t[x].rev ^= 1;
  }

  void push(int x) {
    if (!t[x].rev) return;
    flip(t[x].ch[0]);
    flip(t[x].ch[1]);
    t[x].rev = false;
  }

  void rotate(int x) {
    int p = t[x].p, g = t[p].p, d = (t[p].ch[1] == x);
    if (!isRoot(p)) t[g].ch[t[g].ch[1] == p] = x;
    t[x].p = g;
    t[p].ch[d] = t[x].ch[d ^ 1];
    if (t[x].ch[d ^ 1]) t[t[x].ch[d ^ 1]].p = p;
    t[x].ch[d ^ 1] = p;
    t[p].p = x;
    pull(p);
  }

  void splay(int x) {
    vi stk;
    for (int y = x;; y = t[y].p) {
      stk.push_back(y);
      if (isRoot(y)) break;
    }
    while (!stk.empty()) {
      push(stk.back());
      stk.pop_back();
    }
    while (!isRoot(x)) {
      int p = t[x].p;
      if (!isRoot(p)) {
        int g = t[p].p;
        rotate(((t[g].ch[1] == p) == (t[p].ch[1] == x)) ? p :
     x);
      }
      rotate(x);
    }
    pull(x);
  }

  void access(int x) {
    int last = 0;
    for (int y = x; y; y = t[y].p) {
      splay(y);
      t[y].ch[1] = last;
      pull(y);
      last = y;
    }
    splay(x);
  }

  void makeRoot(int x) {
    access(x);
    flip(x);
  }

  int findRoot(int x) {
    access(x);
    while (true) {
      push(x);
      if (!t[x].ch[0]) break;
      x = t[x].ch[0];
    }
    splay(x);
    return x;
  }

  bool connected(int x, int y) {
    return findRoot(x) == findRoot(y);
  }

  bool link(int x, int y) {
    makeRoot(x);
    if (findRoot(y) == x) return false;
    t[x].p = y;
    return true;
  }

  bool cut(int x, int y) {  // удаляет ребро x-y, если оно ест
     ь
    makeRoot(x);
    access(y);
    if (t[y].ch[0] != x || t[x].ch[0] || t[x].ch[1]) return
     false;
    t[y].ch[0] = 0;
    t[x].p = 0;
    pull(y);
    return true;
  }

  void set(int x, ll v) {
    access(x);
    t[x].val = v;
    pull(x);
  }

  ll pathSum(int x, int y) {  // x, y в одном дереве
    makeRoot(x);
    access(y);
    return t[y].sum;
  }
};
```

Алгоритм Мо

```
// Алгоритм Мо: оффлайн запросы [l, r) к массиву. add(i)/del(i
    ) -- добавить/убрать элемент,
// get() -- ответ на текущем отрезке. O((n + q) sqrt n).
template <typename Add, typename Del, typename Get>
vll mo(int n, const ve<pii> &qs, Add add, Del del, Get get) {
  int q = qs.size(), B = max(1, (int)(n / sqrt((double)max(q,
     1))));
  vi ord(q);
  iota(all(ord), 0);
  sort(all(ord), [&](int a, int b) {
    int ba = qs[a].first / B, bb = qs[b].first / B;
    if (ba != bb) return ba < bb;
    return (ba & 1) ? qs[a].second > qs[b].second : qs[a].
     second < qs[b].second;
  });
  vll res(q);
  int l = 0, r = 0;
  for (int id : ord) {
    while (l > qs[id].first) add(--l);
    while (r < qs[id].second) add(r++);
    while (l < qs[id].first) del(l++);
    while (r > qs[id].second) del(--r);
    res[id] = get();
  }
  return res;
}
```

Динамический KXT

```
// Динамический KXT: добавить прямую y = k * x + m, максимум в точке x. O(log n).
// Для минимума добавлять (-k, -m) и менять знак ответа.
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line &o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  static const ll inf = LLONG_MAX;

  ll div(ll a, ll b) {  // floor(a / b)
    return a / b - ((a ^ b) < 0 && a % b);
  }

  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = (x->m > y->m) ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }

  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p) isect(x,
     erase(y));
  }

  ll query(ll x) {
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

Li Chao

```
// Дерево Ли Чао: максимум прямых в целых точках x in [0, n).
    Прямые можно добавлять
// в любом порядке. Модифицируется до отрезков (add на [lo, hi
    )) обходом как в ДО.
struct LiChao {
  struct Line {
    ll k = 0, b = -INFLL;
    ll operator()(ll x) const { return k * x + b; }
  };

  int n;
  ve<Line> t;

  LiChao() {}
  LiChao(int _n) : n(_n), t(4 * _n + 4) {}

  void add(Line ln, int v, int l, int r) {  // [l, r)
    int mid = (l + r) / 2;
    bool lef = ln(l) > t[v](l), md = ln(mid) > t[v](mid);
    if (md) swap(t[v], ln);
    if (r - l == 1) return;
    if (lef != md) add(ln, 2 * v, l, mid);
    else add(ln, 2 * v + 1, mid, r);
  }
  void add(ll k, ll b) {
    Line ln;
    ln.k = k;
    ln.b = b;
    add(ln, 1, 0, n);
  }

  ll get(int x) {
    ll res = -INFLL;
    int v = 1, l = 0, r = n;
    while (true) {
      res = max(res, t[v](x));
      if (r - l == 1) break;
      int mid = (l + r) / 2;
      if (x < mid) v = 2 * v, r = mid;
      else v = 2 * v + 1, l = mid;
    }
    return res;
  }
};
```

WHO'S GONNA CARRY THE BOATS AND THE LOGS?

3. Графы

Диниц, Гомори–Ху, Штор–Вагнер

```
// Диниц, O(V^2 E) в общем случае, O(E sqrt V) на единичных сетях. Ребро e и обратное e ^ 1.
struct Dinic {
  struct Edge {
    int to;
    ll cap;
  };

  int n;
  ve<Edge> e;
  ve<vi> g;
  vi lvl, ptr;

  Dinic() {}
  Dinic(int _n) : n(_n), g(_n) {}

  int addEdge(int a, int b, ll c, ll rc = 0) {  // rc = c для неориентированного
    g[a].push_back(e.size());
    e.push_back({b, c});
    g[b].push_back(e.size());
    e.push_back({a, rc});
    return (int)e.size() - 2;
  }

  bool bfs(int s, int t) {
    lvl.assign(n, -1);
    lvl[s] = 0;
    queue<int> q;
    q.push(s);
    while (!q.empty()) {
      int v = q.front();
      q.pop();
      for (int id : g[v]) {
        if (e[id].cap > 0 && lvl[e[id].to] == -1) {
          lvl[e[id].to] = lvl[v] + 1;
          q.push(e[id].to);
        }
      }
    }
    return lvl[t] != -1;
  }

  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int &i = ptr[v]; i < (int)g[v].size(); i++) {
      int id = g[v][i], u = e[id].to;
      if (lvl[u] != lvl[v] + 1 || e[id].cap <= 0) continue;
      ll d = dfs(u, t, min(f, e[id].cap));
      if (d) {
        e[id].cap -= d;
        e[id ^ 1].cap += d;
        return d;
      }
    }
    return 0;
  }

  ll maxflow(int s, int t) {
    ll flow = 0;
    while (bfs(s, t)) {
      ptr.assign(n, 0);
      while (ll f = dfs(s, t, INFLL)) flow += f;
    }
    return flow;
  }

  bool inS(int v) {  // после maxflow: v достижима из s в остаточной сети (мин. разрез)
    return lvl[v] != -1;
  }
};

// Дерево Гомори-Ху (Гусфилд): ребро (i, par[i]) веса w[i]. Мин. разрез (u, v) = min на пути.
// edges: неориентированные рёбра (a, b, c).
pair<vi, vll> gomoryHu(int n, const ve<array<ll, 3>> &edges) {
  Dinic base(n);
  for (auto [a, b, c] : edges) base.addEdge(a, b, c, c);
  vi par(n, 0);
  vll w(n, 0);
  for (int i = 1; i < n; i++) {
    Dinic d = base;
    w[i] = d.maxflow(i, par[i]);
    for (int j = i + 1; j < n; j++) {
      if (d.inS(j) && par[j] == par[i]) par[j] = i;
    }
  }
  return {par, w};
}

// Глобальный минимальный разрез (Штор-Вагнер), O(n^3), матрица весов w. Возвращает вес.
ll stoerWagner(ve<vll> w) {
  int n = w.size();
  ll best = INFLL;
  vi vs(n);
  iota(all(vs), 0);
  while (vs.size() > 1) {
    int k = vs.size();
    vll key(k, 0);
    vb used(k, false);
    int prev = -1, last = -1;
    for (int it = 0; it < k; it++) {
      int sel = -1;
      for (int i = 0; i < k; i++) {
        if (!used[i] && (sel == -1 || key[i] > key[sel])) sel
     = i;
      }
```

```
      used[sel] = true;
      prev = last;
      last = sel;
      if (it == k - 1) best = min(best, key[sel]);
      else for (int i = 0; i < k; i++) if (!used[i]) key[i] +=
      w[vs[sel]][vs[i]];
    }
    // сливаем последнюю с предпоследней
    int a = vs[prev], b = vs[last];
    for (int i = 0; i < n; i++) w[a][i] += w[b][i], w[i][a] =
     w[a][i];
    w[a][a] = 0;
    vs.erase(vs.begin() + last);
  }
  return best;
}
```

Mincost maxflow

```
// Мин-кост макс-флоу (SPFA), допускает отрицательные стоимост
     и без отрицательных циклов.
struct MCMF {
  struct Edge {
    int to;
    ll cap, cost;
  };

  int n;
  ve<Edge> e;
  ve<vi> g;

  MCMF(int _n) : n(_n), g(_n) {}

  void addEdge(int a, int b, ll cap, ll cost) {
    g[a].push_back(e.size());
    e.push_back({b, cap, cost});
    g[b].push_back(e.size());
    e.push_back({a, 0, -cost});
  }

  // {поток, стоимость}; пускает не больше maxf
  pll run(int s, int t, ll maxf = INFLL) {
    ll flow = 0, cost = 0;
    while (flow < maxf) {
      vll dist(n, INFLL);
      vi par(n, -1);
      vb inq(n, false);
      deque<int> q;
      dist[s] = 0;
      q.push_back(s);
      while (!q.empty()) {
        int v = q.front();
        q.pop_front();
        inq[v] = false;
        for (int id : g[v]) {
          if (e[id].cap > 0 && dist[e[id].to] > dist[v] + e[id
    ].cost) {
            dist[e[id].to] = dist[v] + e[id].cost;
            par[e[id].to] = id;
            if (!inq[e[id].to]) {
              inq[e[id].to] = true;
              q.push_back(e[id].to);
            }
          }
        }
      }
      if (dist[t] == INFLL) break;
      ll f = maxf - flow;
      for (int v = t; v != s; v = e[par[v] ^ 1].to) f = min(f,
      e[par[v]].cap);
      for (int v = t; v != s; v = e[par[v] ^ 1].to) {
        e[par[v]].cap -= f;
        e[par[v] ^ 1].cap += f;
      }
      flow += f;
      cost += f * dist[t];
    }
    return {flow, cost};
  }
};
```

Паросочетания: Хопкрофт–Карп, блоссом, венгерка

```
// Хопкрофт-Карп: доли [0, n) и [0, m), O(E sqrt V). После
     solve есть вершинное покрытие (Кёниг).
struct HopcroftKarp {
  int n, m;
  ve<vi> g;
  vi ml, mr, dist;

  HopcroftKarp(int _n, int _m) : n(_n), m(_m), g(_n), ml(_n,
    -1), mr(_m, -1) {}

  void addEdge(int l, int r) {
    g[l].push_back(r);
  }

  bool bfs() {
    dist.assign(n, -1);
    queue<int> q;
    for (int v = 0; v < n; v++) if (ml[v] == -1) dist[v] = 0,
     q.push(v);
    bool found = false;
    while (!q.empty()) {
      int v = q.front();
      q.pop();
      for (int u : g[v]) {
        int w = mr[u];
        if (w == -1) found = true;
        else if (dist[w] == -1) dist[w] = dist[v] + 1, q.push(
     w);
      }
    }
    return found;
  }

  bool dfs(int v) {
    for (int u : g[v]) {
      int w = mr[u];
      if (w == -1 || (dist[w] == dist[v] + 1 && dfs(w))) {
        ml[v] = u;
        mr[u] = v;
        return true;
      }
    }
    dist[v] = -1;
    return false;
  }

  int solve() {
    int res = 0;
    while (bfs()) {
      for (int v = 0; v < n; v++) if (ml[v] == -1 && dfs(v))
       res++;
    }
    return res;
  }
};

// Паросочетание в общем графе (Эдмондс, стягивание цветков),
     O(V^3)
struct Blossom {
  int n;
  ve<vi> g;
  vi match, p, base, q;
  vb used, blossom;

  Blossom(int _n) : n(_n), g(_n), match(_n, -1) {}

  void addEdge(int a, int b) {
    g[a].push_back(b);
    g[b].push_back(a);
  }

  int lca(int a, int b) {
    vb u(n, false);
    while (true) {
      a = base[a];
      u[a] = true;
      if (match[a] == -1) break;
      a = p[match[a]];
    }
    while (true) {
      b = base[b];
      if (u[b]) return b;
      b = p[match[b]];
    }
  }

  void markPath(int v, int b, int child) {
    while (base[v] != b) {
      blossom[base[v]] = blossom[base[match[v]]] = true;
      p[v] = child;
      child = match[v];
      v = p[match[v]];
    }
  }

  int findPath(int root) {
    used.assign(n, false);
    // min вершинное покрытие: (левые, правые); независимое множ
       ество -- дополнение
    pair<vi, vi> cover() {
      vb visL(n, false), visR(m, false);
      vi st;
      for (int v = 0; v < n; v++) if (ml[v] == -1) visL[v] =
       true, st.push_back(v);
      while (!st.empty()) {
        int v = st.back();
        st.pop_back();
        for (int u : g[v]) {
          if (visR[u]) continue;
          visR[u] = true;
          int w = mr[u];
          if (w != -1 && !visL[w]) visL[w] = true, st.push_back(
       w);
        }
      }
      vi L, R;
      for (int v = 0; v < n; v++) if (!visL[v]) L.push_back(v);
      for (int u = 0; u < m; u++) if (visR[u]) R.push_back(u);
      return {L, R};
    }
};
```

```
    p.assign(n, -1);
    base.resize(n);
    iota(all(base), 0);
    used[root] = true;
    q.assign(1, root);
    for (size_t qh = 0; qh < q.size(); qh++) {
      int v = q[qh];
      for (int to : g[v]) {
        if (base[v] == base[to] || match[v] == to) continue;
        if (to == root || (match[to] != -1 && p[match[to]] !=
     -1)) {
          int cur = lca(v, to);
          blossom.assign(n, false);
          markPath(v, cur, to);
          markPath(to, cur, v);
          for (int i = 0; i < n; i++) {
            if (!blossom[base[i]]) continue;
            base[i] = cur;
            if (!used[i]) used[i] = true, q.push_back(i);
          }
        } else if (p[to] == -1) {
          p[to] = v;
          if (match[to] == -1) return to;
          used[match[to]] = true;
          q.push_back(match[to]);
        }
      }
    }
    return -1;
  }

  int solve() {
    int res = 0;
    for (int i = 0; i < n; i++) {
      if (match[i] != -1) continue;
      int v = findPath(i);
      if (v == -1) continue;
      res++;
      while (v != -1) {
        int pv = p[v], ppv = match[pv];
        match[v] = pv;
        match[pv] = v;
        v = ppv;
      }
    }
    return res;
  }
};

// Венгерский алгоритм: a[1..n][1..m], n <= m, минимизирует ст
     оимость. O(n^2 m)
// Возвращает {стоимость, ans[i] = столбец строки i}
pair<ll, vi> hungarian(const ve<vll> &a, int n, int m) {
  vll u(n + 1), v(m + 1);
  vi p(m + 1), way(m + 1);
  for (int i = 1; i <= n; i++) {
    p[0] = i;
    int j0 = 0;
    vll minv(m + 1, INFLL);
    vb used(m + 1, false);
    do {
      used[j0] = true;
      int i0 = p[j0], j1 = 0;
      ll delta = INFLL;
      for (int j = 1; j <= m; j++) {
        if (used[j]) continue;
        ll cur = a[i0][j] - u[i0] - v[j];
        if (cur < minv[j]) minv[j] = cur, way[j] = j0;
        if (minv[j] < delta) delta = minv[j], j1 = j;
      }
      for (int j = 0; j <= m; j++) {
        if (used[j]) u[p[j]] += delta, v[j] -= delta;
        else minv[j] -= delta;
      }
      j0 = j1;
    } while (p[j0] != 0);
    do {
      int j1 = way[j0];
      p[j0] = p[j1];
      j0 = j1;
    } while (j0);
  }
  vi ans(n + 1);
  for (int j = 1; j <= m; j++) if (p[j]) ans[p[j]] = j;
  return {-v[0], ans};
}
```

KCC и 2-SAT

```
// Тарьян: comp[v] -- номер KCC, номера идут в обратном тополо
     гическом порядке
// (для ребра u -> v в разные KCC comp[u] > comp[v]). Рекурсия
     : при глубине ~1e6 нужен ulimit -s.
struct SCC {
  int n, timer = 0, cnt = 0;
  ve<vi> g;
  vi tin, low, comp, stk;
  vb on;

  SCC(int _n) : n(_n), g(_n), tin(_n, -1), low(_n), comp(_n,
     -1), on(_n, false) {}

  void addEdge(int a, int b) {
    g[a].push_back(b);
  }

  void dfs(int v) {
    tin[v] = low[v] = timer++;
    stk.push_back(v);
    on[v] = true;
    for (int u : g[v]) {
      if (tin[u] == -1) {
        dfs(u);
        low[v] = min(low[v], low[u]);
      } else if (on[u]) {
        low[v] = min(low[v], tin[u]);
      }
    }
    if (low[v] == tin[v]) {
      while (true) {
        int u = stk.back();
        stk.pop_back();
        on[u] = false;
        comp[u] = cnt;
        if (u == v) break;
      }
      cnt++;
    }
  }

  int run() {
    for (int v = 0; v < n; v++) if (tin[v] == -1) dfs(v);
    return cnt;
  }
};

// 2-SAT: переменная i -- литералы 2i (истина) и 2i + 1 (ложь)
     ; lit ^ 1 -- отрицание.
struct TwoSat {
  int n;
  SCC s;
  vb val;

  TwoSat(int _n) : n(_n), s(2 * _n) {}

  void addClause(int a, int b) {  // (a или b)
    s.addEdge(a ^ 1, b);
    s.addEdge(b ^ 1, a);
  }

  bool solve() {
    s.run();
    val.assign(n, false);
    for (int i = 0; i < n; i++) {
      if (s.comp[2 * i] == s.comp[2 * i + 1]) return false;
      val[i] = s.comp[2 * i] < s.comp[2 * i + 1];
    }
    return true;
  }
};
```

Мосты, точки сочленения, BCC

```
// Мосты, точки сочленения и вершинные компоненты двусвязности
     . Мультиребра допустимы.
struct Bridges {
  int n, timer = 0;
  ve<ve<pii>> g;  // (куда, номер ребра)
  vi tin, low, stk;
  vb isCut, isBridge;
  ve<vi> bcc;  // вершинные компоненты (каждая -- список верши
     н)

  Bridges(int _n, const ve<pii> &edges) : n(_n), g(_n), tin(_n
     , -1), low(_n), isCut(_n, false),
            isBridge(edges.size(), false) {
    for (int i = 0; i < (int)edges.size(); i++) {
      g[edges[i].first].push_back({edges[i].second, i});
      g[edges[i].second].push_back({edges[i].first, i});
    }
    for (int v = 0; v < n; v++) if (tin[v] == -1) {
      dfs(v, -1, true);
    }
  }

  void dfs(int v, int pe, bool root) {
    tin[v] = low[v] = timer++;
    stk.push_back(v);
    int ch = 0;
    for (auto [u, id] : g[v]) {
      if (id == pe) continue;
      if (tin[u] != -1) {
        low[v] = min(low[v], tin[u]);
        continue;
      }
      dfs(u, id, false);
      ch++;
      low[v] = min(low[v], low[u]);
      if (low[u] > tin[v]) isBridge[id] = true;
      if (low[u] >= tin[v]) {
        if (!root) isCut[v] = true;
        bcc.push_back({v});
        while (true) {
          int w = stk.back();
```

```
          stk.pop_back();
          bcc.back().push_back(w);
          if (w == u) break;
        }
      }
    }
    if (root && ch > 1) isCut[v] = true;
    if (root && ch == 0) bcc.push_back({v});
  }
};
```

HLD и центроидная декомпозиция

```
// HLD: корень 0. pos[v] -- позиция в массиве, поддерево v = [
    pos[v], pos[v] + sz[v]).
// path(u, v) -- отрезки позиций [l, r] (вершинные значения),
    lca за O(log n).
struct HLD {
  int n;
  vec<vi> g;
  vi par, dep, sz, heavy, head, pos;

  HLD(int _n) : n(_n), g(_n), par(_n, -1), dep(_n), sz(_n),
     heavy(_n, -1), head(_n), pos(_n) {}

  void addEdge(int a, int b) {
    g[a].push_back(b);
    g[b].push_back(a);
  }

  void dfs(int v) {
    sz[v] = 1;
    for (int u : g[v]) {
      if (u == par[v]) continue;
      par[u] = v;
      dep[u] = dep[v] + 1;
      dfs(u);
      sz[v] += sz[u];
      if (heavy[v] == -1 || sz[u] > sz[heavy[v]]) heavy[v] = u
    ;
    }
  }

  void build(int root = 0) {
    dfs(root);
    int cur = 0;
    function<void(int, int)> go = [&](int v, int h) {
      head[v] = h;
      pos[v] = cur++;
      if (heavy[v] != -1) go(heavy[v], h);
      for (int u : g[v]) if (u != par[v] && u != heavy[v]) go(
    u, u);
    };
    go(root, root);
  }

  int lca(int a, int b) {
    while (head[a] != head[b]) {
      if (dep[head[a]] < dep[head[b]]) swap(a, b);
      a = par[head[a]];
    }
    return dep[a] < dep[b] ? a : b;
  }

  vec<pii> path(int a, int b) {  // отрезки позиций, покрывающи
     е путь a..b
    vec<pii> res;
    while (head[a] != head[b]) {
      if (dep[head[a]] < dep[head[b]]) swap(a, b);
      res.push_back({pos[head[a]], pos[a]});
      a = par[head[a]];
    }
    res.push_back({min(pos[a], pos[b]), max(pos[a], pos[b])});
    return res;
  }
};

// Центроидная декомпозиция: cpar -- родитель в дереве центрои
    дов, clvl -- глубина, O(n log n)
struct Centroid {
  int n;
  vec<vi> g;
  vi cpar, clvl, sz;
  vb removed;

  Centroid(int _n) : n(_n), g(_n), cpar(_n, -1), clvl(_n, 0),
     sz(_n), removed(_n, false) {}

  void addEdge(int a, int b) {
    g[a].push_back(b);
    g[b].push_back(a);
  }

  int calc(int v, int p) {
    sz[v] = 1;
    for (int u : g[v]) if (u != p && !removed[u]) sz[v] +=
     calc(u, v);
    return sz[v];
  }

  int findC(int v, int p, int tot) {
    for (int u : g[v]) if (u != p && !removed[u] && sz[u] * 2
     > tot) return findC(u, v, tot);
    return v;
  }

  void build(int v = 0, int p = -1, int lvl = 0) {
    int tot = calc(v, -1);
    int c = findC(v, -1, tot);
    cpar[c] = p;
    clvl[c] = lvl;
    removed[c] = true;
    for (int u : g[c]) if (!removed[u]) build(u, c, lvl + 1);
  }
};
```

Оффлайн динамическая связность

```
// + dsu.cpp

// Оффлайн динамическая связность: рёбра живут на отрезке врем
    ени [l, r). Дерево отрезков по
// времени + СНМ с откатами, O((n + m) log T log n). run() --
    число компонент в каждый момент.
struct OfflineConn {
  int n, T, comps;
  vec<ve<pii>> seg;
  DSU d;
  vi ans;

  OfflineConn(int _n, int _T) : n(_n), T(_T), comps(_n), seg(4
     * _T + 4), d(_n), ans(_T) {}

  void addEdge(int l, int r, int u, int v, int node, int tl,
     int tr) {
    if (r <= tl || tr <= l) return;
    if (l <= tl && tr <= r) {
      seg[node].push_back({u, v});
      return;
    }
    int mid = (tl + tr) / 2;
    addEdge(l, r, u, v, 2 * node, tl, mid);
    addEdge(l, r, u, v, 2 * node + 1, mid, tr);
  }

  void addEdge(int l, int r, int u, int v) {
    addEdge(l, r, u, v, 1, 0, T);
  }

  void rec(int node, int tl, int tr) {
    int snap = d.snapshot(), cc = comps;
    for (auto [a, b] : seg[node]) if (d.unite(a, b)) comps--;
    if (tr - tl == 1) ans[tl] = comps;
    else {
      int mid = (tl + tr) / 2;
      rec(2 * node, tl, mid);
      rec(2 * node + 1, mid, tr);
    }
    d.rollback(snap);
    comps = cc;
  }

  vi run() {
    rec(1, 0, T);
    return ans;
  }
};
```

Пути, Эйлеров путь, направленный остов

```
// Дейкстра, O(E log V). g[v] = (u, w), w >= 0
vll dijkstra(const vec<vec<pair<int, ll>>> &g, int s) {
  vll d(g.size(), INFLL);
  priority_queue<pll, vec<pll>, greater<pll>> pq;
  d[s] = 0;
  pq.push({0, s});
  while (!pq.empty()) {
    auto [dv, v] = pq.top();
    pq.pop();
    if (dv != d[v]) continue;
    for (auto [u, w] : g[v]) {
      if (d[u] > dv + w) {
        d[u] = dv + w;
        pq.push({d[u], u});
      }
    }
  }
  return d;
}

// Форд-Беллман: false, если из s достижим отрицательный цикл.
    edges (a, b, w)
bool bellmanFord(int n, const vec<array<ll, 3>> &edges, int s,
    vll &d) {
  d.assign(n, INFLL);
  d[s] = 0;
  for (int it = 0; it < n; it++) {
    bool ch = false;
    for (auto [a, b, w] : edges) {
      if (d[a] != INFLL && d[b] > d[a] + w) {
        d[b] = d[a] + w;
        ch = true;
      }
    }
```

```
    if (!ch) return true;
  }
  return false;
}

void floyd(ve<vll> &d) {  // d[i][j] = INFLL, если ребра нет; d[i][i] = 0
  int n = d.size();
  for (int k = 0; k < n; k++) for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) {
    if (d[i][k] < INFLL && d[k][j] < INFLL) d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
  }
}

// Эйлеров путь/цикл (Хирхольцер). edges (a, b); возвращает вершины пути (m + 1 штук) или пусто.
vi eulerPath(int n, const ve<pii> &edges, bool directed) {
  int m = edges.size();
  ve<vi> g(n);
  vi deg(n, 0), in(n, 0);
  for (int i = 0; i < m; i++) {
    auto [a, b] = edges[i];
    g[a].push_back(i);
    if (directed) in[b]++;
    else g[b].push_back(i);
    deg[a]++;
    if (!directed) deg[b]++;
  }
  int start = -1, odd = 0, cs = 0, ce = 0;
  for (int v = 0; v < n; v++) {
    if (directed) {
      int d = deg[v] - in[v];
      if (d == 1) cs++, start = v;
      else if (d == -1) ce++;
      else if (d != 0) return {};
    } else if (deg[v] & 1) {
      odd++;
      start = v;
    }
  }
  if (!directed && odd > 2) return {};
  if (directed && (cs > 1 || ce > 1 || cs != ce)) return {};
  if (start == -1) for (int v = 0; v < n; v++) if (!g[v].empty()) { start = v; break; }
  if (start == -1) return {0};
  vb used(m, false);
  vi ptr(n, 0), st = {start}, res;
  while (!st.empty()) {
    int v = st.back();
    while (ptr[v] < (int)g[v].size() && used[g[v][ptr[v]]]) ptr[v]++;
    if (ptr[v] == (int)g[v].size()) {
      res.push_back(v);
      st.pop_back();
    } else {
      int id = g[v][ptr[v]];
      used[id] = true;
      st.push_back(edges[id].first == v ? edges[id].second : edges[id].first);
    }
  }
  if ((int)res.size() != m + 1) return {};
  reverse(all(res));
  return res;
}

// Минимальный остов ориентированного графа с корнем root (Чу-Лю/Эдмондс), O(VE).
// edges (u, v, w): u -> v. Возвращает вес или -1.
ll dirMST(int n, int root, ve<array<ll, 3>> edges) {
  ll res = 0;
  while (true) {
    vll in(n, INFLL);
    vi pre(n, -1);
    for (auto [u, v, w] : edges) if (u != v && w < in[v]) in[v] = w, pre[v] = u;
    for (int i = 0; i < n; i++) if (i != root && in[i] == INFLL) return -1;
    int cnt = 0;
    vi id(n, -1), vis(n, -1);
    in[root] = 0;
    for (int v = 0; v < n; v++) {
      res += in[v];
      int u = v;
      while (vis[u] != v && id[u] == -1 && u != root) vis[u] = v, u = pre[u];
      if (u != root && id[u] == -1) {
        for (int x = pre[u]; x != u; x = pre[x]) id[x] = cnt;
        id[u] = cnt++;
      }
    }
    if (!cnt) break;
    for (int v = 0; v < n; v++) if (id[v] == -1) id[v] = cnt++;
    for (auto &e : edges) {
      ll old = e[1];
      e[0] = id[e[0]], e[1] = id[e[1]];
      if (e[0] != e[1]) e[2] -= in[old];
    }
    n = cnt;
    root = id[root];
  }
  return res;
}
```

THEY DON'T KNOW US, SON.

4. Математика

Теория чисел: gcd, КТО, решето

```
ll binpow(ll a, ll b, ll m = MOD) {  // a * a должно влезать в ll (m < 3e9)
  ll r = 1 % m;
  for (a %= m, a += (a < 0) * m; b; b >>= 1, a = a * a % m) {
    if (b & 1) r = r * a % m;
  }
  return r;
}

ll extgcd(ll a, ll b, ll &x, ll &y) {  // a * x + b * y = gcd(a, b)
  if (!b) {
    x = 1, y = 0;
    return a;
  }
  ll g = extgcd(b, a % b, y, x);
  y -= a / b * x;
  return g;
}

ll inv(ll a, ll m = MOD) {  // обратный по любому модулю, gcd(a, m) = 1
  ll x, y;
  extgcd((a % m + m) % m, m, x, y);
  return (x % m + m) % m;
}

// a * x + b * y = c. Возвращает false, если решений нет. Остальные: (x + k * b / g, y - k * a / g).
bool diophant(ll a, ll b, ll c, ll &x, ll &y) {
  ll g = extgcd(a, b, x, y);
  if (c % g) return false;
  x *= c / g;
  y *= c / g;
  return true;
}

// x = a1 (mod m1), x = a2 (mod m2) -> (x mod lcm, lcm) или (-1, -1)
pll crt(ll a1, ll m1, ll a2, ll m2) {
  ll x, y, g = extgcd(m1, m2, x, y);
  if ((a2 - a1) % g) return {-1, -1};
  ll l = m1 / g * m2, m = m2 / g;
  ll k = ((a2 - a1) / g % m + m) % m * (x % m + m) % m % m;
  ll res = (ll)(((__int128)m1 * k + a1) % l);
  return {(res + l) % l, l};
}

struct Sieve {  // линейное решето: lp, простые, phi, mu
  vi lp, primes, phi, mu;

  Sieve(int n) : lp(n + 1), phi(n + 1), mu(n + 1) {
    phi[1] = mu[1] = 1;
    for (int i = 2; i <= n; i++) {
      if (!lp[i]) {
        lp[i] = i;
        phi[i] = i - 1;
        mu[i] = -1;
        primes.push_back(i);
      }
      for (int p : primes) {
        if (p > lp[i] || (ll)i * p > n) break;
        lp[i * p] = p;
        if (i % p == 0) {
          phi[i * p] = phi[i] * p;
          mu[i * p] = 0;
        } else {
          phi[i * p] = phi[i] * (p - 1);
          mu[i * p] = -mu[i];
        }
      }
    }
  }
};

ll phiFunc(ll n) {  // O(sqrt n)
  ll res = n;
  for (ll p = 2; p * p <= n; p++) {
    if (n % p) continue;
    while (n % p == 0) n /= p;
    res -= res / p;
  }
  if (n > 1) res -= res / n;
  return res;
}
```

Миллер–Рабин, Ро-Поллард

```
ull mulmod(ull a, ull b, ull m) {
  return (unsigned __int128)a * b % m;
```

```
}

ull powmod(ull a, ull b, ull m) {
  ull r = 1 % m;
  for (; b; b >>= 1, a = mulmod(a, a, m)) {
    if (b & 1) r = mulmod(r, a, m);
  }
  return r;
}

// Детерминированный Миллер-Рабин для всех n < 2^64
bool isPrime(ull n) {
  if (n < 2) return false;
  for (ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
    if (n % p == 0) return n == p;
  }
  ull d = n - 1;
  int s = 0;
  while (d % 2 == 0) d /= 2, s++;
  for (ull a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
    ull x = powmod(a, d, n);
    if (x == 1 || x == n - 1) continue;
    bool comp = true;
    for (int i = 1; i < s && comp; i++) {
      x = mulmod(x, x, n);
      if (x == n - 1) comp = false;
    }
    if (comp) return false;
  }
  return true;
}

// Ро-Полларда (Флойд, gcd раз в 64 шага; при g = n откат и gcd
    на каждом шаге)
ull rho(ull n) {
  if (n % 2 == 0) return 2;
  static mt19937_64 gen(2024);
  while (true) {
    ull c = gen() % (n - 1) + 1, x = gen() % n, y = x, g = 1;
    auto f = [&](ull v) {
      ull r = mulmod(v, v, n) + c;
      if (r < c || r >= n) r -= n;
      return r;
    };
    auto step = [&]() {
      x = f(x);
      y = f(f(y));
      return x > y ? x - y : y - x;
    };
    while (g == 1) {
      ull sx = x, sy = y, prod = 1;
      for (int i = 0; i < 64; i++) prod = mulmod(prod, step(),
     n);
      g = __gcd(prod, n);
      if (g == n) {
        x = sx, y = sy;
        for (g = 1; g == 1;) g = __gcd(step(), n);  // d = 0
     -> gcd = n
      }
    }
    if (g != n) return g;
  }
}

void factorRec(ull n, ve<ull> &out) {
  if (n == 1) return;
  if (isPrime(n)) {
    out.push_back(n);
    return;
  }
  ull d = rho(n);
  factorRec(d, out);
  factorRec(n / d, out);
}

ve<ull> factor(ull n) {  // простые делители с кратностью, по
    возрастанию
  ve<ull> res;
  for (ull p = 2; p < 100 && p * p <= n; p++) {
    while (n % p == 0) res.push_back(p), n /= p;
  }
  factorRec(n, res);
  sort(all(res));
  return res;
}
```

sqrt, первообразный корень, лог, floor-sum

```
// + numth.cpp

// sqrt(a) mod p, p простое (Тонелли-Шенкс), -1 если корня нет
ll sqrtMod(ll a, ll p) {
  a %= p;
  if (a < 0) a += p;
  if (a == 0 || p == 2) return a;
  if (binpow(a, (p - 1) / 2, p) != 1) return -1;
  if (p % 4 == 3) return binpow(a, (p + 1) / 4, p);
  ll q = p - 1;
  int s = 0;
  while (q % 2 == 0) q /= 2, s++;
  ll z = 2;
  while (binpow(z, (p - 1) / 2, p) != p - 1) z++;
  ll c = binpow(z, q, p), x = binpow(a, (q + 1) / 2, p), t =
     binpow(a, q, p);
  int m = s;
  while (t != 1) {
    int i = 0;
    for (ll tt = t; tt != 1; tt = tt * tt % p) i++;
    ll b = c;
    for (int j = 0; j < m - i - 1; j++) b = b * b % p;
    x = x * b % p;
    c = b * b % p;
    t = t * c % p;
    m = i;
  }
  return x;
}

// Первообразный корень по простому p
ll primRoot(ll p) {
  if (p == 2) return 1;
  ve<ll> fs;
  ll n = p - 1;
  for (ll d = 2; d * d <= n; d++) {
    if (n % d) continue;
    fs.push_back(d);
    while (n % d == 0) n /= d;
  }
  if (n > 1) fs.push_back(n);
  for (ll g = 2;; g++) {
    bool ok = true;
    for (ll f : fs) ok &= binpow(g, (p - 1) / f, p) != 1;
    if (ok) return g;
  }
}

// min x >= 0: a^x = b (mod m), любой m; -1 если нет. O(sqrt m
    log m)
ll modlog(ll a, ll b, ll m) {
  a %= m, b %= m;
  ll k = 1 % m, add = 0, g;
  while ((g = __gcd(a, m)) > 1) {
    if (b == k) return add;
    if (b % g) return -1;
    b /= g, m /= g, add++;
    k = k * (a / g) % m;
  }
  ll n = (ll)sqrt((ld)m) + 1, an = 1;
  for (ll i = 0; i < n; i++) an = an * a % m;
  map<ll, ll> vals;
  for (ll q = 0, cur = b; q <= n; q++, cur = cur * a % m) vals
    [cur] = q;
  for (ll p = 1, cur = k; p <= n; p++) {
    cur = cur * an % m;
    auto it = vals.find(cur);
    if (it != vals.end()) return n * p - it->second + add;
  }
  return -1;
}

// sum_{i=0}^{n-1} floor((a * i + b) / m), a, b >= 0
ll floorSum(ll n, ll m, ll a, ll b) {
  ll res = 0;
  while (true) {
    if (a >= m) res += (n - 1) * n / 2 * (a / m), a %= m;
    if (b >= m) res += n * (b / m), b %= m;
    ll y = a * n + b;
    if (y < m) break;
    n = y / m;
    b = y % m;
    swap(m, a);
  }
  return res;
}
```

Суммы по делителям, Мёбиус, НОД/НОК-свёртки

```
// + numth.cpp

// Суммы по делителям/кратным за O(n log log n) (по простым),
    индексы 1..n (f[0] не используется).
// zetaDiv:  g[m] = sum_{d | m} f[d];      mobDiv: обратная (М
    ёбиус по делителям)
// zetaMul:  g[d] = sum_{d | m} f[m];      mobMul: обратная (М
    ёбиус по кратным)
void zetaDiv(vll &f, const vi &primes) {
  int n = (int)f.size() - 1;
  for (int p : primes) for (int i = 1; i * p <= n; i++) f[i *
    p] += f[i];
}

void mobDiv(vll &f, const vi &primes) {
  int n = (int)f.size() - 1;
  for (int p : primes) for (int i = n / p; i >= 1; i--) f[i *
    p] -= f[i];
}

void zetaMul(vll &f, const vi &primes) {
  int n = (int)f.size() - 1;
  for (int p : primes) for (int i = n / p; i >= 1; i--) f[i]
    += f[i * p];
}

void mobMul(vll &f, const vi &primes) {
```

```
  int n = (int)f.size() - 1;
  for (int p : primes) for (int i = 1; i * p <= n; i++) f[i]
     -= f[i * p];
}

// c[k] = sum_{gcd(i, j) = k} a[i] b[j]
vll gcdConv(vll a, vll b, const vi &primes) {
  zetaMul(a, primes);
  zetaMul(b, primes);
  for (size_t i = 0; i < a.size(); i++) a[i] *= b[i];
  mobMul(a, primes);
  return a;
}

// c[k] = sum_{lcm(i, j) = k} a[i] b[j]  (пары с lcm > n отбра
     сываются)
vll lcmConv(vll a, vll b, const vi &primes) {
  zetaDiv(a, primes);
  zetaDiv(b, primes);
  for (size_t i = 0; i < a.size(); i++) a[i] *= b[i];
  mobDiv(a, primes);
  return a;
}

// Свёртка Дирихле c[n] = sum_{ij = n} a[i] b[j], O(n log n)
vll dirichlet(const vll &a, const vll &b) {
  int n = (int)a.size() - 1;
  vll c(n + 1, 0);
  for (int i = 1; i <= n; i++) for (int j = 1; i * j <= n; j
     ++) c[i * j] += a[i] * b[j];
  return c;
}
```

Гаусс, определитель, матричная степень

```
// + numth.cpp

// Расширенная матрица a (n x (m + 1)) по простому модулю p. В
     озвращает ранг матрицы
// коэффициентов, или -1 если система несовместна. x -- одно р
     ешение (свободные = 0).
int gaussMod(ve<vll> a, vll &x, ll p = MOD) {
  int n = a.size(), m = a[0].size() - 1, rk = 0;
  vi where(m, -1);
  for (int col = 0; col < m && rk < n; col++) {
    int sel = -1;
    for (int i = rk; i < n; i++) if (a[i][col] % p) { sel = i;
      break; }
    if (sel == -1) continue;
    swap(a[sel], a[rk]);
    where[col] = rk;
    ll iv = inv(a[rk][col], p);
    for (auto &v : a[rk]) v = v * iv % p;
    for (int i = 0; i < n; i++) {
      if (i == rk || !a[i][col]) continue;
      ll c = a[i][col];
      for (int j = col; j <= m; j++) a[i][j] = ((a[i][j] - c *
      a[rk][j]) % p + p) % p;
    }
    rk++;
  }
  x.assign(m, 0);
  for (int i = 0; i < m; i++) if (where[i] != -1) x[i] = a[
     where[i]][m];
  for (int i = rk; i < n; i++) if (a[i][m] % p) return -1;
  return rk;
}

ll detMod(ve<vll> a, ll p = MOD) {
  int n = a.size();
  ll det = 1;
  for (int col = 0; col < n; col++) {
    int sel = -1;
    for (int i = col; i < n; i++) if (a[i][col] % p) { sel = i
     ; break; }
    if (sel == -1) return 0;
    if (sel != col) swap(a[sel], a[col]), det = (p - det) % p;
    det = det * a[col][col] % p;
    ll iv = inv(a[col][col], p);
    for (int i = col + 1; i < n; i++) {
      ll c = a[i][col] * iv % p;
      for (int j = col; j < n; j++) a[i][j] = ((a[i][j] - c *
     a[col][j]) % p + p) % p;
    }
  }
  return det;
}

// Гаусс по вещественным: расширенная матрица, частичный выбор
     ведущего. Возвращает ранг / -1.
int gaussD(ve<ve<ld>> a, ve<ld> &x, ld eps = 1e-9) {
  int n = a.size(), m = a[0].size() - 1, rk = 0;
  vi where(m, -1);
  for (int col = 0; col < m && rk < n; col++) {
    int sel = rk;
    for (int i = rk; i < n; i++) if (abs(a[i][col]) > abs(a[
     sel][col])) sel = i;
    if (abs(a[sel][col]) < eps) continue;
    swap(a[sel], a[rk]);
    where[col] = rk;
    for (int i = 0; i < n; i++) {
      if (i == rk) continue;
      ld c = a[i][col] / a[rk][col];
      for (int j = col; j <= m; j++) a[i][j] -= c * a[rk][j];
    }
    rk++;
  }
  x.assign(m, 0);
  for (int i = 0; i < m; i++) if (where[i] != -1) x[i] = a[
     where[i]][m] / a[where[i]][i];
  for (int i = rk; i < n; i++) if (abs(a[i][m]) > eps) return
     -1;
  return rk;
}

using Mat = ve<vll>;

Mat matMul(const Mat &a, const Mat &b, ll p = MOD) {
  int n = a.size(), m = b[0].size(), k = b.size();
  Mat c(n, vll(m));
  for (int i = 0; i < n; i++) for (int t = 0; t < k; t++) for
     (int j = 0; j < m; j++) {
    c[i][j] = (c[i][j] + a[i][t] * b[t][j]) % p;
  }
  return c;
}

Mat matPow(Mat a, ll e, ll p = MOD) {
  int n = a.size();
  Mat r(n, vll(n));
  for (int i = 0; i < n; i++) r[i][i] = 1;
  for (; e; e >>= 1, a = matMul(a, a, p)) if (e & 1) r =
     matMul(r, a, p);
  return r;
}
```

Берлекэмп–Мэсси, n-й член, Лагранж

```
// + numth.cpp

// Берлекэмп-Мэсси по модулю MOD (простой): s[i] = sum_j c[j]
     * s[i - 1 - j].
vll berlekampMassey(const vll &s) {
  int n = s.size(), L = 0, m = 0;
  vll C(n), B(n), T;
  C[0] = B[0] = 1;
  ll b = 1;
  for (int i = 0; i < n; i++) {
    m++;
    ll d = s[i] % MOD;
    for (int j = 1; j <= L; j++) d = (d + C[j] * s[i - j]) %
     MOD;
    if (!d) continue;
    T = C;
    ll coef = d * inv(b) % MOD;
    for (int j = m; j < n; j++) C[j] = ((C[j] - coef * B[j - m
     ]) % MOD + MOD) % MOD;
    if (2 * L > i) continue;
    L = i + 1 - L, B = T, b = d, m = 0;
  }
  C.resize(L + 1);
  C.erase(C.begin());
  for (auto &x : C) x = (MOD - x) % MOD;
  return C;
}

// k-й член (с нуля) по рекурренте c и начальным s: x^k mod xa
     рактеристический многочлен. O(n^2 log k)
ll nthTerm(const vll &s, const vll &c, ll k) {
  int n = c.size();
  if (k < (ll)s.size()) return s[k];
  auto mul = [&](const vll &a, const vll &b) {
    vll r(2 * n, 0);
    for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) r[
     i + j] = (r[i + j] + a[i] * b[j]) % MOD;
    for (int i = 2 * n - 1; i >= n; i--) {
      for (int j = 0; j < n; j++) r[i - 1 - j] = (r[i - 1 - j]
     + r[i] * c[j]) % MOD;
    }
    r.resize(n);
    return r;
  };
  vll res(n, 0), base(n, 0);
  res[0] = 1;
  if (n == 1) base[0] = c[0];
  else base[1] = 1;
  for (; k; k >>= 1, base = mul(base, base)) if (k & 1) res =
     mul(res, base);
  ll ans = 0;
  for (int i = 0; i < n; i++) ans = (ans + res[i] * s[i]) %
     MOD;
  return ans;
}

// y[i] = f(i), i = 0..n-1, deg f < n. f(x) mod MOD за O(n)
ll lagrange(const vll &y, ll x) {
  int n = y.size();
  x %= MOD;
  if (x < n) return y[x];
  vll pre(n + 1, 1), suf(n + 1, 1), fact(n, 1);
  for (int i = 0; i < n; i++) pre[i + 1] = pre[i] * ((x - i +
     MOD) % MOD) % MOD;
  for (int i = n - 1; i >= 0; i--) suf[i] = suf[i + 1] * ((x -
      i + MOD) % MOD) % MOD;
```

```
  for (int i = 1; i < n; i++) fact[i] = fact[i - 1] * i % MOD;
  ll res = 0;
  for (int i = 0; i < n; i++) {
    ll term = y[i] * pre[i] % MOD * suf[i + 1] % MOD;
    term = term * inv(fact[i] * fact[n - 1 - i] % MOD) % MOD;
    if ((n - 1 - i) & 1) term = (MOD - term) % MOD;
    res = (res + term) % MOD;
  }
  return res;
}
```

Комбинаторика, Лукас, pi(n)

```
// + numth.cpp

// Факториалы по модулю MOD до n. C, A (размещения), Каталан.
struct Combi {
  vll f, fi;

  Combi(int n) : f(n + 1), fi(n + 1) {
    f[0] = 1;
    for (int i = 1; i <= n; i++) f[i] = f[i - 1] * i % MOD;
    fi[n] = inv(f[n]);
    for (int i = n; i > 0; i--) fi[i - 1] = fi[i] * i % MOD;
  }

  ll C(int n, int k) {
    if (k < 0 || k > n) return 0;
    return f[n] * fi[k] % MOD * fi[n - k] % MOD;
  }

  ll A(int n, int k) {
    if (k < 0 || k > n) return 0;
    return f[n] * fi[n - k] % MOD;
  }

  ll catalan(int n) {
    return C(2 * n, n) * inv(n + 1) % MOD;
  }
};

// C(n, k) mod p для малого простого p (Лукас): O(p + log_p n)
ll lucas(ll n, ll k, int p) {
  static vll f;
  static int fp = 0;
  if (fp != p) {
    f.assign(p, 1);
    for (int i = 1; i < p; i++) f[i] = f[i - 1] * i % p;
    fp = p;
  }
  ll res = 1;
  for (; n || k; n /= p, k /= p) {
    int a = n % p, b = k % p;
    if (b > a) return 0;
    res = res * f[a] % p * inv(f[b] * f[a - b] % p, p) % p;
  }
  return res;
}

// Число простых <= n (Люси-Хаджеаз), O(n^{3/4})
ll primeCount(ll n) {
  if (n < 2) return 0;
  ll r = sqrtl(n);
  while (r * r > n) r--;
  while ((r + 1) * (r + 1) <= n) r++;
  vll lo(r + 2), hi(r + 2);
  for (ll i = 1; i <= r; i++) lo[i] = i - 1, hi[i] = n / i -
    1;
  for (ll p = 2; p <= r; p++) {
    if (lo[p] == lo[p - 1]) continue;
    ll pc = lo[p - 1], p2 = p * p, end = min(r, n / p2);
    for (ll i = 1; i <= end; i++) {
      ll d = i * p;
      hi[i] -= (d <= r ? hi[d] : lo[n / d]) - pc;
    }
    for (ll i = r; i >= p2; i--) lo[i] -= lo[i / p] - pc;
  }
  return hi[1];
}
```

Симплекс

```
// Симплекс: max c^T x, A x <= b, x >= 0. solve возвращает зна
    чение (или +-INF если неограничено /
// нет решений: inf / -inf), x -- оптимальная точка. Сложность
     на практике быстрая, худший случай
// экспоненциальный. Для равенств -- два неравенства. Правило
    Бланда по (значение, номер).
struct LPSolver {
  int m, n;
  vi N, B;
  ve<ve<ld>> D;
  static constexpr ld e = 1e-9, inf = 1e18;

  LPSolver(const ve<ve<ld>> &A, const ve<ld> &b, const ve<ld>
     &c)
    : m(b.size()), n(c.size()), N(n + 1), B(m), D(m + 2, ve<ld
    >(n + 2)) {
    for (int i = 0; i < m; i++) for (int j = 0; j < n; j++) D[
    i][j] = A[i][j];
    for (int i = 0; i < m; i++) B[i] = n + i, D[i][n] = -1, D[
    i][n + 1] = b[i];
    for (int j = 0; j < n; j++) N[j] = j, D[m][j] = -c[j];
    N[n] = -1;
    D[m + 1][n] = 1;
  }

  void pivot(int r, int s) {
    ld iv = 1 / D[r][s];
    for (int i = 0; i < m + 2; i++) {
      if (i == r || abs(D[i][s]) <= e) continue;
      ld k = D[i][s] * iv;
      for (int j = 0; j < n + 2; j++) D[i][j] -= D[r][j] * k;
      D[i][s] = D[r][s] * k;
    }
    for (int j = 0; j < n + 2; j++) if (j != s) D[r][j] *= iv;
    for (int i = 0; i < m + 2; i++) if (i != r) D[i][s] *= -iv
    ;
    D[r][s] = iv;
    swap(B[r], N[s]);
  }

  bool simplex(int phase) {
    int x = m + phase - 1;
    while (true) {
      int s = -1;
      for (int j = 0; j <= n; j++) {
        if (N[j] == -phase) continue;
        if (s == -1 || make_pair(D[x][j], N[j]) < make_pair(D[
   x][s], N[s])) s = j;
      }
      if (D[x][s] >= -e) return true;
      int r = -1;
      for (int i = 0; i < m; i++) {
        if (D[i][s] <= e) continue;
        if (r == -1 || make_pair(D[i][n + 1] / D[i][s], B[i])
     < make_pair(D[r][n + 1] / D[r][s], B[r])) r = i;
      }
      if (r == -1) return false;
      pivot(r, s);
    }
  }

  ld solve(ve<ld> &x) {
    int r = 0;
    for (int i = 1; i < m; i++) if (D[i][n + 1] < D[r][n + 1])
      r = i;
    if (D[r][n + 1] < -e) {
      pivot(r, n);
      if (!simplex(2) || D[m + 1][n + 1] < -e) return -inf;
      for (int i = 0; i < m; i++) if (B[i] == -1) {
        int s = 0;
        for (int j = 1; j <= n; j++) {
          if (make_pair(D[i][j], N[j]) < make_pair(D[i][s], N[
    s])) s = j;
        }
        pivot(i, s);
      }
    }
    bool ok = simplex(1);
    x.assign(n, 0);
    for (int i = 0; i < m; i++) if (B[i] < n) x[B[i]] = D[i][n
      + 1];
    return ok ? D[m][n + 1] : inf;
  }
};
```

STAY FOCUSED. STAY FUCKING HARD.

5. Преобразования и оптимизации

NTT и свёртка по модулю

```
// + numth.cpp

// NTT по модулю простого M = c * 2^k + 1 с первообразным корн
    ем G.
// 998244353 (g = 3), 167772161 (g = 3), 469762049 (g = 3).
template<int M, int G>
void ntt(ve<int> &a, bool invert) {
  int n = a.size();
  for (int i = 1, j = 0; i < n; i++) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) swap(a[i], a[j]);
  }
  for (int len = 2; len <= n; len <<= 1) {
    ll w = binpow(G, (M - 1) / len, M);
    if (invert) w = binpow(w, M - 2, M);
    for (int i = 0; i < n; i += len) {
      ll cur = 1;
      for (int j = 0; j < len / 2; j++) {
        int u = a[i + j], v = a[i + j + len / 2] * cur % M;
        a[i + j] = (u + v >= M) ? u + v - M : u + v;
        a[i + j + len / 2] = (u - v < 0) ? u - v + M : u - v;
        cur = cur * w % M;
      }
    }
  }
  if (invert) {
    ll ni = binpow(n, M - 2, M);
    for (auto &x : a) x = x * ni % M;
```

```
  }
}

template<int M, int G>
ve<int> convNTT(ve<int> a, ve<int> b) {
  int need = a.size() + b.size() - 1, n = 1;
  while (n < need) n <<= 1;
  a.resize(n), b.resize(n);
  for (auto &x : a) x %= M;
  for (auto &x : b) x %= M;
  ntt<M, G>(a, false);
  ntt<M, G>(b, false);
  for (int i = 0; i < n; i++) a[i] = (ll)a[i] * b[i] % M;
  ntt<M, G>(a, true);
  a.resize(need);
  return a;
}

ve<int> conv(const ve<int> &a, const ve<int> &b) {  // mod 998244353
  return convNTT<998244353, 3>(a, b);
}

// Свёртка по произвольному модулю mod (< 2^31): три NTT-модуля + Гарнер.
// Коэффициенты в [0, mod), длина до ~ 1e6.
ve<int> convMod(const ve<int> &a, const ve<int> &b, int mod) {
  const int m1 = 998244353, m2 = 167772161, m3 = 469762049;
  auto r1 = convNTT<m1, 3>(a, b), r2 = convNTT<m2, 3>(a, b),
    r3 = convNTT<m3, 3>(a, b);
  ll i12 = inv(m1, m2), i123 = inv((ll)m1 * m2 % m3, m3);
  ve<int> res(r1.size());
  for (size_t i = 0; i < res.size(); i++) {
    ll x2 = ((r2[i] - r1[i]) % m2 + m2) % m2 * i12 % m2;
    ll x12 = r1[i] + x2 * m1;  // < m1 * m2
    ll x3 = ((r3[i] - x12 % m3) % m3 + m3) % m3 * i123 % m3;
    res[i] = (x12 % mod + x3 * ((ll)m1 * m2 % mod)) % mod;
  }
  return res;
}
```

FFT (double)

```
using cd = complex<double>;
const double FPI = acos(-1.0);

void fft(ve<cd> &a, bool invert) {
  int n = a.size();
  for (int i = 1, j = 0; i < n; i++) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) swap(a[i], a[j]);
  }
  ve<cd> rt(n / 2 + 1);
  for (int i = 0; i < n / 2; i++) rt[i] = polar(1.0, 2 * FPI *
      i / n * (invert ? -1 : 1));
  for (int len = 2; len <= n; len <<= 1) {
    int step = n / len;
    for (int i = 0; i < n; i += len) {
      for (int j = 0; j < len / 2; j++) {
        cd u = a[i + j], v = a[i + j + len / 2] * rt[j * step
    ];
        a[i + j] = u + v;
        a[i + j + len / 2] = u - v;
      }
    }
  }
  if (invert) for (auto &x : a) x /= n;
}

// Точная целочисленная свёртка, пока |ответ| <~ 1e14 / log n
    (для больших -- разбивать на 15 бит)
ve<ll> convFFT(const ve<ll> &a, const ve<ll> &b) {
  int need = a.size() + b.size() - 1, n = 1;
  while (n < need) n <<= 1;
  ve<cd> fa(all(a)), fb(all(b));
  fa.resize(n), fb.resize(n);
  fft(fa, false);
  fft(fb, false);
  for (int i = 0; i < n; i++) fa[i] *= fb[i];
  fft(fa, true);
  ve<ll> res(need);
  for (int i = 0; i < need; i++) res[i] = llround(fa[i].real()
    );
  return res;
}
```

Многочлены: inv, log, exp, Стирлинг-строки

```
// + ntt.cpp

// Многочлены по модулю 998244353 (вектор коэффициентов по возрастанию степени).
const int PM = 998244353;
using poly = ve<int>;

poly polyMul(const poly &a, const poly &b) {
  return conv(a, b);
}

poly polyInv(const poly &a, int n) {  // 1 / a mod x^n, a[0] != 0; Ньютон: b = b (2 - a b)
  poly b = {(int)inv(a[0], PM)};
  for (int m = 1; m < n; m *= 2) {
    poly t(a.begin(), a.begin() + min((int)a.size(), 2 * m));
    poly ab = conv(t, b);
    ab.resize(2 * m);
    for (auto &x : ab) x = (x == 0) ? 0 : PM - x;
    ab[0] = (ab[0] + 2) % PM;
    b = conv(b, ab);
    b.resize(2 * m);
  }
  b.resize(n);
  return b;
}

poly polyDer(const poly &a) {
  poly r(max<int>(0, (int)a.size() - 1));
  for (size_t i = 1; i < a.size(); i++) r[i - 1] = (ll)a[i] *
    i % PM;
  return r;
}

poly polyInt(const poly &a) {
  poly r(a.size() + 1);
  for (size_t i = 0; i < a.size(); i++) r[i + 1] = a[i] * inv(
    i + 1, PM) % PM;
  return r;
}

poly polyLog(const poly &a, int n) {  // ln a mod x^n, a[0] = 1
  poly r = conv(polyDer(a), polyInv(a, n));
  r.resize(n - 1);
  r = polyInt(r);
  r.resize(n);
  return r;
}

poly polyExp(const poly &a, int n) {  // exp a mod x^n, a[0] = 0; Ньютон: b = b (1 - ln b + a)
  poly b = {1};
  for (int m = 1; m < n; m *= 2) {
    poly l = polyLog(b, 2 * m);
    for (int i = 0; i < 2 * m; i++) {
      int ai = i < (int)a.size() ? a[i] : 0;
      l[i] = ((ai - l[i]) % PM + PM) % PM;
    }
    l[0] = (l[0] + 1) % PM;
    b = conv(b, l);
    b.resize(2 * m);
  }
  b.resize(n);
  return b;
}

// c(n, k): коэффициенты x (x + 1) ... (x + n - 1) -- беззнаковые числа Стирлинга I рода, D&C + NTT
poly prodLin(int l, int r) {  // prod_{t = l}^{r - 1} (x + t)
  if (r - l == 1) return {l % PM, 1};
  int m = (l + r) / 2;
  return conv(prodLin(l, m), prodLin(m, r));
}

poly stirling1Row(int n) {
  return n == 0 ? poly{1} : prodLin(0, n);
}

// S(n, k), k = 0..n : [x^k] (sum i^n x^i / i!) (sum (-1)^j x^j / j!)
poly stirling2Row(int n) {
  ve<ll> f(n + 1, 1);
  for (int i = 1; i <= n; i++) f[i] = f[i - 1] * i % PM;
  poly a(n + 1), b(n + 1);
  for (int i = 0; i <= n; i++) {
    ll fi = inv(f[i], PM);
    a[i] = binpow(i, n, PM) * fi % PM;
    b[i] = (i & 1) ? (PM - fi) % PM : fi;
  }
  poly r = conv(a, b);
  r.resize(n + 1);
  return r;
}
```

Уолш–Адамар, SOS

```
// Преобразование Уолша-Адамара. type: 0 -- xor, 1 -- and (суперсеты), 2 -- or (подмножества).
// Свёртка: f(a), f(b) -> покомпонентное произведение -> f^{-1} (invert = true). Длина = 2^k.
void fwht(vll &a, int type, bool invert) {
  int n = a.size();
  for (int len = 1; len < n; len <<= 1) {
    for (int i = 0; i < n; i += 2 * len) {
      for (int j = i; j < i + len; j++) {
        ll u = a[j], v = a[j + len];
        if (type == 0) a[j] = u + v, a[j + len] = u - v;
        else if (type == 1) a[j] = invert ? u - v : u + v;
        else a[j + len] = invert ? v - u : u + v;
      }
    }
  }
  if (type == 0 && invert) for (auto &x : a) x /= n;
```

```
}

vll xorConv(vll a, vll b, int type = 0) {
  fwht(a, type, false);
  fwht(b, type, false);
  for (size_t i = 0; i < a.size(); i++) a[i] *= b[i];
  fwht(a, type, true);
  return a;
}

// SOS-dp: f[mask] = сумма a[sub] по подмножествам; обратное
    -- вычитание
void sosSum(vll &f, int n) {
  for (int i = 0; i < n; i++) {
    for (int mask = 0; mask < (1 << n); mask++) {
      if (mask >> i & 1) f[mask] += f[mask ^ (1 << i)];
    }
  }
}

void sosInv(vll &f, int n) {
  for (int i = 0; i < n; i++) {
    for (int mask = 0; mask < (1 << n); mask++) {
      if (mask >> i & 1) f[mask] -= f[mask ^ (1 << i)];
    }
  }
}
```

D&C, Кнут, НВП

```
// Оптимизация D&C: dp[j] = min_{k < j} prev[k] + cost(k, j),
    opt(j) монотонен
// (достаточно неравенства четырёхугольника на cost). O(n log
    n) на слой.
// prev[k] = INFLL -- недостижимо. Слои: for (i) cur = dcLayer
    (prev, cost).
template <typename F>
void dcRec(const vll &prev, vll &cur, int l, int r, int optl,
    int optr, F &cost) {
  if (l > r) return;
  int mid = (l + r) / 2, opt = -1;
  ll best = INFLL;
  for (int k = optl; k <= min(mid - 1, optr); k++) {
    if (prev[k] >= INFLL) continue;
    ll v = prev[k] + cost(k, mid);
    if (v < best) best = v, opt = k;
  }
  cur[mid] = best;
  if (opt == -1) opt = optl;
  dcRec(prev, cur, l, mid - 1, optl, opt, cost);
  dcRec(prev, cur, mid + 1, r, opt, optr, cost);
}

template <typename F>
vll dcLayer(const vll &prev, F cost) {
  int n = prev.size();
  vll cur(n, INFLL);
  dcRec(prev, cur, 0, n - 1, 0, n - 1, cost);
  return cur;
}

// Кнут: dp[l][r] = min_{l <= k < r} dp[l][k] + dp[k + 1][r] +
    cost(l, r), dp[i][i] = 0.
// Нужны QI и монотонность cost по вложению. O(n^2). Возвращае
    т dp[0][n - 1].
template <typename F>
ll knuth(int n, F cost) {
  ve<vll> dp(n, vll(n, 0));
  ve<vi> opt(n, vi(n, 0));
  for (int i = 0; i < n; i++) opt[i][i] = i;
  for (int len = 2; len <= n; len++) {
    for (int l = 0; l + len <= n; l++) {
      int r = l + len - 1;
      dp[l][r] = INFLL;
      for (int k = opt[l][r - 1]; k <= min(r - 1, opt[l + 1][r
   ]); k++) {
        ll v = dp[l][k] + dp[k + 1][r] + cost(l, r);
        if (v < dp[l][r]) dp[l][r] = v, opt[l][r] = k;
      }
    }
  }
  return dp[0][n - 1];
}

// НВП (строго возрастающая): длина и сама подпоследовательнос
    ть (индексы), O(n log n)
vi lis(const vi &a) {
  int n = a.size();
  vi d, idx, par(n, -1);
  for (int i = 0; i < n; i++) {
    int p = lower_bound(all(d), a[i], [&](int j, int x) {
     return a[j] < x; }) - d.begin();
    if (p == (int)d.size()) d.push_back(i);
    else d[p] = i;
    par[i] = p ? d[p - 1] : -1;
  }
  vi res;
  for (int v = d.empty() ? -1 : d.back(); v != -1; v = par[v])
      res.push_back(v);
  reverse(all(res));
  return res;
}
```

CALLUS YOUR MIND.

6. Строки

Z, префикс, Манакер, суфмас, хеш

```
// z[i] = lcp(s, s[i..]), z[0] = 0
vi z_func(const string &s) {
  int n = s.length();
  vi z(n);
  int l = 0, r = 0;
  for (int i = 1; i < n; i++) {
    if (i < r) z[i] = min(r - i, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] > r) l = i, r = i + z[i];
  }
  return z;
}

// p[i] = длина наибольшего собственного префикса-суффикса s
    [0..i]
vi pref_func(const string &s) {
  int n = s.length();
  vi p(n);
  for (int i = 1; i < n; i++) {
    int j = p[i - 1];
    while (j > 0 && s[j] != s[i]) j = p[j - 1];
    if (s[i] == s[j]) j++;
    p[i] = j;
  }
  return p;
}

// Манакер: d1[i] -- число нечётных палиндромов с центром i (р
    адиус), d2[i] -- число чётных
// палиндромов с центром между i - 1 и i. O(n)
pair<vi, vi> manacher(const string &s) {
  int n = s.size();
  vi d1(n), d2(n);
  for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
    while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k
     ++;
    d1[i] = k--;
    if (i + k > r) l = i - k, r = i + k;
  }
  for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
    while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i
     + k]) k++;
    d2[i] = k--;
    if (i + k > r) l = i - k - 1, r = i + k;
  }
  return {d1, d2};
}

// Начало лексикографически минимального циклического сдвига,
    O(n)
int minRotation(const string &s) {
  int n = s.size(), i = 0, j = 1, k = 0;
  while (i < n && j < n && k < n) {
    char a = s[(i + k) % n], b = s[(j + k) % n];
    if (a == b) {
      k++;
      continue;
    }
    if (a > b) i += k + 1;
    else j += k + 1;
    if (i == j) j++;
    k = 0;
  }
  return min(i, j);
}

// Полиномиальный хеш по модулю 2^61 - 1 со случайной базой.
    get(l, r) -- хеш s[l, r)
struct Hash61 {
  static const ull M = (1ULL << 61) - 1;
  vector<ull> h, pw;

  static ull mul(ull a, ull b) {
    unsigned __int128 t = (unsigned __int128)a * b;
    ull r = (ull)(t & M) + (ull)(t >> 61);
    return r >= M ? r - M : r;
  }

  Hash61(const string &s) : h(s.size() + 1), pw(s.size() + 1)
    {
    static const ull base = mt19937_64(chrono::steady_clock::
     now().time_since_epoch().count())() % (M - 1000) + 500;
    pw[0] = 1;
    for (size_t i = 0; i < s.size(); i++) {
      pw[i + 1] = mul(pw[i], base);
      h[i + 1] = mul(h[i], base) + (ull)s[i];
      if (h[i + 1] >= M) h[i + 1] -= M;
    }
  }

  ull get(int l, int r) {
    ull res = h[r] + M - mul(h[l], pw[r - l]);
    return res >= M ? res - M : res;
```

```
  }
};

// Суффиксный массив (удвоение + сортировка подсчётом), O(n
    log n). Возвращает sa.
vi suffixArray(string s) {
  s += char(1);
  int n = s.size(), cls = 1;
  vi p(n), c(n), cnt(max(256, n), 0);
  for (char ch : s) cnt[(unsigned char)ch]++;
  for (int i = 1; i < 256; i++) cnt[i] += cnt[i - 1];
  for (int i = 0; i < n; i++) p[--cnt[(unsigned char)s[i]]] =
     i;
  c[p[0]] = 0;
  for (int i = 1; i < n; i++) {
    if (s[p[i]] != s[p[i - 1]]) cls++;
    c[p[i]] = cls - 1;
  }
  vi pn(n), cn(n);
  for (int h = 0; (1 << h) < n; h++) {
    for (int i = 0; i < n; i++) {
      pn[i] = p[i] - (1 << h);
      if (pn[i] < 0) pn[i] += n;
    }
    fill(cnt.begin(), cnt.begin() + cls, 0);
    for (int i = 0; i < n; i++) cnt[c[pn[i]]]++;
    for (int i = 1; i < cls; i++) cnt[i] += cnt[i - 1];
    for (int i = n - 1; i >= 0; i--) p[--cnt[c[pn[i]]]] = pn[i
    ];
    cn[p[0]] = 0;
    cls = 1;
    for (int i = 1; i < n; i++) {
      pii cur = {c[p[i]], c[(p[i] + (1 << h)) % n]};
      pii prv = {c[p[i - 1]], c[(p[i - 1] + (1 << h)) % n]};
      if (cur != prv) cls++;
      cn[p[i]] = cls - 1;
    }
    c.swap(cn);
  }
  p.erase(p.begin());
  return p;
}

// lcp[i] = lcp(suffix sa[i], suffix sa[i + 1]) (Касай), разме
    p n - 1
vi lcpArray(const string &s, const vi &sa) {
  int n = s.size(), k = 0;
  vi rk(n), lcp(max(n - 1, 0));
  for (int i = 0; i < n; i++) rk[sa[i]] = i;
  for (int i = 0; i < n; i++) {
    if (rk[i] == n - 1) {
      k = 0;
      continue;
    }
    int j = sa[rk[i] + 1];
    while (i + k < n && j + k < n && s[i + k] == s[j + k]) k
    ++;
    lcp[rk[i]] = k;
    if (k) k--;
  }
  return lcp;
}
```

Автомат, Ахо-Корасик, эертри

```
const int ALPHA = 26;  // алфавит 'a'..'z'

// Суффиксный автомат: len, link, переходы next. Различных под
    строк = sum(len[v] - len[link[v]]).
struct SufAuto {
  struct State {
    int len = 0, link = -1;
    array<int, ALPHA> next;
    State() { next.fill(-1); }
  };

  ve<State> st;
  int last = 0;

  SufAuto() : st(1) {}
  SufAuto(const string &s) : st(1) {
    for (char c : s) add(c - 'a');
  }

  void add(int c) {
    int cur = st.size();
    st.emplace_back();
    st[cur].len = st[last].len + 1;
    int p = last;
    while (p != -1 && st[p].next[c] == -1) {
      st[p].next[c] = cur;
      p = st[p].link;
    }
    if (p == -1) st[cur].link = 0;
    else {
      int q = st[p].next[c];
      if (st[p].len + 1 == st[q].len) st[cur].link = q;
      else {
        int cl = st.size();
        st.push_back(st[q]);
        st[cl].len = st[p].len + 1;
        while (p != -1 && st[p].next[c] == q) {
          st[p].next[c] = cl;
          p = st[p].link;
        }
        st[q].link = st[cur].link = cl;
      }
    }
    last = cur;
  }

  ll distinct() {
    ll res = 0;
    for (size_t v = 1; v < st.size(); v++) res += st[v].len -
     st[st[v].link].len;
    return res;
  }
};

// Ахо-Корасик: add(слово) -> id; после build() go[v][c] -- по
    лный автомат, cnt[v] -- число
// слов-суффиксов состояния (суммируется по суффиксным ссылкам
    ).
struct AhoCorasick {
  ve<array<int, ALPHA>> go;
  vi link, cnt;

  AhoCorasick() {
    newNode();
  }

  int newNode() {
    go.emplace_back();
    go.back().fill(-1);
    link.push_back(0);
    cnt.push_back(0);
    return (int)go.size() - 1;
  }

  int add(const string &s) {
    int v = 0;
    for (char ch : s) {
      int c = ch - 'a';
      if (go[v][c] == -1) {
        int u = newNode();
        go[v][c] = u;
      }
      v = go[v][c];
    }
    cnt[v]++;
    return v;
  }

  void build() {
    queue<int> q;
    for (int c = 0; c < ALPHA; c++) {
      if (go[0][c] == -1) go[0][c] = 0;
      else q.push(go[0][c]);
    }
    while (!q.empty()) {
      int v = q.front();
      q.pop();
      cnt[v] += cnt[link[v]];
      for (int c = 0; c < ALPHA; c++) {
        int u = go[v][c];
        if (u == -1) go[v][c] = go[link[v]][c];
        else {
          link[u] = go[link[v]][c];
          q.push(u);
        }
      }
    }
  }
};

// Palindromic tree (eertree): вершины 0 (len -1, корень нечёт
    ных), 1 (len 0), дальше -- палиндромы.
// Различных палиндромов-подстрок = size - 2. occ[v] после
    finish() -- число вхождений.
struct Eertree {
  struct Node {
    int len, link;
    array<int, ALPHA> next;
    ll occ = 0;
  };

  ve<Node> t;
  string s;
  int last = 1;

  Eertree() {
    t.push_back({-1, 0, {}, 0});
    t.push_back({0, 0, {}, 0});
    t[0].next.fill(-1);
    t[1].next.fill(-1);
  }

  int getLink(int v) {
    int i = s.size() - 1;
    while (i - t[v].len - 1 < 0 || s[i - t[v].len - 1] != s[i
    ]) v = t[v].link;
    return v;
  }
```

```
  void add(char ch) {
    s += ch;
    int c = ch - 'a', cur = getLink(last);
    if (t[cur].next[c] == -1) {
      int v = t.size();
      t.push_back({t[cur].len + 2, 0, {}, 0});
      t[v].next.fill(-1);
      if (t[v].len == 1) t[v].link = 1;
      else t[v].link = t[getLink(t[cur].link)].next[c];
      t[cur].next[c] = v;
    }
    last = t[cur].next[c];
    t[last].occ++;
  }

  void finish() {  // накопить вхождения по суффиксным ссылкам
    for (int v = (int)t.size() - 1; v > 1; v--) t[t[v].link].
     occ += t[v].occ;
  }
};
```

NO ONE IS COMING TO SAVE YOU. SOLVE IT.

7. Геометрия

База: точка, прямая, окружность

```
// База геометрии (стиль: mpt/mln/mcr + псевдонимы pt/ln/cr).
     Один код для ld и ll:
// sgn сравнивает с eps, для целых это точное сравнение. ll-то
     чки -- для точных предикатов.
const ld eps = 1e-9;
const ld PI = acosl(-1);

template <typename T>
int sgn(T a) {
  return (a > eps) - (a < -eps);
}

bool lt(ld a, ld b) { return sgn(b - a) > 0; }
bool eq(ld a, ld b) { return sgn(a - b) == 0; }

template <typename T>
struct mpt {
  T x, y;
  mpt() : x(0), y(0) {}
  mpt(T _x, T _y) : x(_x), y(_y) {}
  mpt(const mpt &a, const mpt &b) : x(b.x - a.x), y(b.y - a.y)
     {}  // вектор a -> b

  mpt operator+(const mpt &a) const { return mpt(x + a.x, y +
     a.y); }
  mpt operator-(const mpt &a) const { return mpt(x - a.x, y -
     a.y); }
  mpt operator*(T k) const { return mpt(x * k, y * k); }
  mpt operator/(T k) const { return mpt(x / k, y / k); }
  T operator*(const mpt &a) const { return x * a.x + y * a.y;
     }  // скалярное
  T operator%(const mpt &a) const { return x * a.y - y * a.x;
     }  // векторное
  bool operator==(const mpt &a) const { return sgn(x - a.x) ==
      0 && sgn(y - a.y) == 0; }
  bool operator<(const mpt &a) const { return sgn(x - a.x) ? x
      < a.x : sgn(y - a.y) < 0; }

  T len2() const { return x * x + y * y; }
  ld len() const { return sqrtl(len2()); }
  mpt turn90() const { return mpt(-y, x); }
  mpt rot(ld a) const { return mpt(x * cosl(a) - y * sinl(a),
     x * sinl(a) + y * cosl(a)); }
  ld angle(const mpt &a) const { return atan2l(*this % a, *
     this * a); }  // угол от this к a, (-pi, pi]
  ld pol() const { return atan2l(y, x); }
};

using pt = mpt<ld>;
using ipt = mpt<ll>;

template <typename T>
istream& operator>>(istream &is, mpt<T> &a) {
  return is >> a.x >> a.y;
}

template <typename T>
ostream& operator<<(ostream &os, const mpt<T> &a) {
  return os << a.x << ' ' << a.y;
}

// Прямая a x + b y + c = 0. g(p) > 0 справа от направления p
     -> q (конструктор по 2 точкам).
template <typename T>
struct mln {
  T a, b, c;
  mln() : a(0), b(0), c(0) {}
  mln(T _a, T _b, T _c) : a(_a), b(_b), c(_c) {}
  mln(const mpt<T> &t, const mpt<T> &q) {
    mpt<T> p(t, q);
    a = p.y;
    b = -p.x;
    c = -a * q.x - b * q.y;
  }

  T g(const mpt<T> &p) const { return a * p.x + b * p.y + c; }
  mpt<T> nr() const { return mpt<T>(a, b); }   // нормаль
  mpt<T> pr() const { return mpt<T>(-b, a); }  // направление
  ld dist(const mpt<T> &p) const { return fabsl((ld)g(p)) / nr
     ().len(); }
  mpt<T> proj(const mpt<T> &p) const {
    mpt<T> n = nr();
    return p - n * (g(p) / n.len2());
  }
};

using ln = mln<ld>;

// Пересечение непараллельных прямых
pt inter(const ln &l, const ln &m) {
  ld d = l.a * m.b - l.b * m.a;
  return pt((l.b * m.c - l.c * m.b) / d, (l.c * m.a - l.a * m.
     c) / d);
}

bool parallel(const ln &l, const ln &m) {
  return sgn(l.a * m.b - l.b * m.a) == 0;
}

// Окружность. fln/fcr возвращают 0, 1 или 2 точки; совпадающи
     е окружности -> пустой вектор
// (совпадение окружностей проверять снаружи).
template <typename T>
struct mcr {
  mpt<T> c;
  T r;
  mcr() : r(0) {}
  mcr(const mpt<T> &_c, T _r) : c(_c), r(_r) {}
  ve<pt> fln(const ln &l) const {
    ld d = l.dist(c);
    if (lt(r, d)) return {};
    pt p = l.proj(c);
    if (eq(d, r)) return {p};
    pt v = l.pr();
    v = v * (sqrtl(max<ld>(0, r * r - d * d)) / v.len());
    return {p + v, p - v};
  }

  ve<pt> fcr(const mcr &w) const {
    if (c == w.c) return {};
    ln l(2 * (c.x - w.c.x), 2 * (c.y - w.c.y),
         r * r - w.r * w.r + w.c.len2() - c.len2());
    return fln(l);
  }

  ve<pt> tang(const pt &p) const {  // точки касания касательн
     ых из p
    pt v(c, p);
    ld d2 = v.len2();
    if (lt(d2, r * r)) return {};
    if (eq(d2, r * r)) return {p};
    return fcr(mcr<ld>(p, sqrtl(d2 - r * r)));
  }
};

using cr = mcr<ld>;

// Описанная окружность треугольника (не вырожденного)
cr circum(const pt &a, const pt &b, const pt &c) {
  pt ab(a, b), ac(a, c);
  ld d = 2 * (ab % ac);
  pt o(ac.y * ab.len2() - ab.y * ac.len2(), ab.x * ac.len2() -
      ac.x * ab.len2());
  o = o / d;
  return cr(a + o, o.len());
}

template <typename T>
bool onSeg(const mpt<T> &p, const mpt<T> &a, const mpt<T> &b)
     {
  return sgn((a - p) % (b - p)) == 0 && sgn((a - p) * (b - p))
      <= 0;
}

// Пересекаются ли замкнутые отрезки ab и cd
template <typename T>
bool segInter(const mpt<T> &a, const mpt<T> &b, const mpt<T> &
     c, const mpt<T> &d) {
  int d1 = sgn((b - a) % (c - a)), d2 = sgn((b - a) % (d - a))
     ;
  int d3 = sgn((d - c) % (a - c)), d4 = sgn((d - c) % (b - c))
     ;
  if (d1 * d2 < 0 && d3 * d4 < 0) return true;
  return onSeg(c, a, b) || onSeg(d, a, b) || onSeg(a, c, d) ||
      onSeg(b, c, d);
}

ld segDist(const pt &p, const pt &a, const pt &b) {
  pt v(a, b);
  if (sgn(v.len2()) == 0) return pt(p, a).len();
  ld t = ((p - a) * v) / v.len2();
  t = max<ld>(0, min<ld>(1, t));
  return pt(p, a + v * t).len();
}
```

```
// Полярная сортировка: полуплоскость, затем векторное произведение
template <typename T>
int half(const mpt<T> &p) {
  return (sgn(p.y) > 0 || (sgn(p.y) == 0 && sgn(p.x) >= 0)) ? 0 : 1;
}

template <typename T>
bool angLess(const mpt<T> &l, const mpt<T> &r) {
  if (half(l) != half(r)) return half(l) < half(r);
  return sgn(l % r) > 0;
}
```

Оболочка, полигоны, ПП, калиперы, круг и полигон

```
// + point2d.cpp

// Удвоенная ориентированная площадь (> 0 для обхода против часовой)
template <typename T>
T area2(const ve<mpt<T>> &p) {
  T s = 0;
  for (size_t i = 0; i < p.size(); i++) s += p[i] % p[(i + 1) % p.size()];
  return s;
}

// Выпуклая оболочка (Эндрю), ccw, без коллинеарных точек. Дубликаты убираются.
template <typename T>
ve<mpt<T>> hull(ve<mpt<T>> a) {
  sort(all(a));
  a.erase(unique(all(a)), a.end());
  int n = a.size();
  if (n < 3) return a;
  ve<mpt<T>> h(2 * n);
  int k = 0;
  for (int i = 0; i < n; i++) {
    while (k >= 2 && sgn((h[k - 1] - h[k - 2]) % (a[i] - h[k - 1])) <= 0) k--;
    h[k++] = a[i];
  }
  for (int i = n - 2, t = k + 1; i >= 0; i--) {
    while (k >= t && sgn((h[k - 1] - h[k - 2]) % (a[i] - h[k - 1])) <= 0) k--;
    h[k++] = a[i];
  }
  h.resize(k - 1);
  return h;
}

// Точка в произвольном многоугольнике: 0 -- снаружи, 1 -- на границе, 2 -- внутри. O(n)
template <typename T>
int pip(const ve<mpt<T>> &p, const mpt<T> &q) {
  int n = p.size(), in = 0;
  for (int i = 0; i < n; i++) {
    const mpt<T> &a = p[i], &b = p[(i + 1) % n];
    if (onSeg(q, a, b)) return 1;
    if ((sgn(a.y - q.y) > 0) != (sgn(b.y - q.y) > 0)) {
      int s = sgn((b - a) % (q - a));
      if (s == (b.y > a.y ? 1 : -1)) in ^= 1;
    }
  }
  return in ? 2 : 0;
}

// Точка в выпуклом ccw многоугольнике (с границей), O(log n)
template <typename T>
bool pinConvex(const ve<mpt<T>> &p, const mpt<T> &q) {
  int n = p.size();
  if (n == 1) return p[0] == q;
  if (n == 2) return onSeg(q, p[0], p[1]);
  if (sgn((p[1] - p[0]) % (q - p[0])) < 0 || sgn((p[n - 1] - p[0]) % (q - p[0])) > 0) return false;
  int l = 1, r = n - 1;
  while (r - l > 1) {
    int m = (l + r) / 2;
    if (sgn((p[m] - p[0]) % (q - p[0])) >= 0) l = m;
    else r = m;
  }
  return sgn((p[l + 1] - p[l]) % (q - p[l])) >= 0;
}

// Диаметр выпуклого многоугольника (вращающиеся калиперы), квадрат расстояния. h = hull(...)
template <typename T>
T diameter2(const ve<mpt<T>> &h) {
  int n = h.size();
  if (n == 1) return 0;
  if (n == 2) return (h[0] - h[1]).len2();
  T best = 0;
  for (int i = 0, j = 1; i < n; i++) {
    while (sgn((h[(i + 1) % n] - h[i]) % (h[(j + 1) % n] - h[j])) > 0) j = (j + 1) % n;
    best = max(best, max((h[i] - h[j]).len2(), (h[(i + 1) % n] - h[j]).len2()));
  }
  return best;
}

// Сумма Минковского выпуклых ccw многоугольников (hull-формат), O(n + m)
template <typename T>
ve<mpt<T>> minkowski(ve<mpt<T>> a, ve<mpt<T>> b) {
  auto fix = [](ve<mpt<T>> &p) {
    int k = 0;
    for (int i = 1; i < (int)p.size(); i++) {
      if (sgn(p[i].y - p[k].y) < 0 || (sgn(p[i].y - p[k].y) == 0 && sgn(p[i].x - p[k].x) < 0)) k = i;
    }
    rotate(p.begin(), p.begin() + k, p.end());
    p.push_back(p[0]);
    p.push_back(p[1 % (p.size() - 1)]);
  };
  fix(a);
  fix(b);
  ve<mpt<T>> res;
  size_t i = 0, j = 0;
  while (i < a.size() - 2 || j < b.size() - 2) {
    res.push_back(a[i] + b[j]);
    int c = sgn((a[i + 1] - a[i]) % (b[j + 1] - b[j]));
    if (c >= 0 && i < a.size() - 2) i++;
    if (c <= 0 && j < b.size() - 2) j++;
  }
  return res;
}

// Ближайшая пара точек: min квадрат расстояния, n >= 2. O(n log n)
template <typename T>
T closestPair(ve<mpt<T>> p) {
  sort(all(p));
  int n = p.size();
  T best = numeric_limits<T>::max();
  ve<mpt<T>> tmp(n);
  auto byY = [](const mpt<T> &a, const mpt<T> &b) { return a.y < b.y; };
  function<void(int, int)> rec = [&](int l, int r) {
    if (r - l <= 3) {
      for (int i = l; i < r; i++) for (int j = i + 1; j < r; j++) best = min(best, (p[i] - p[j]).len2());
      sort(p.begin() + l, p.begin() + r, byY);
      return;
    }
    int m = (l + r) / 2;
    T midx = p[m].x;
    rec(l, m);
    rec(m, r);
    merge(p.begin() + l, p.begin() + m, p.begin() + m, p.begin() + r, tmp.begin(), byY);
    copy(tmp.begin(), tmp.begin() + (r - l), p.begin() + l);
    int k = 0;
    for (int i = l; i < r; i++) {
      if ((p[i].x - midx) * (p[i].x - midx) >= best) continue;
      for (int j = k - 1; j >= 0 && (p[i].y - tmp[j].y) * (p[i].y - tmp[j].y) < best; j--) {
        best = min(best, (p[i] - tmp[j]).len2());
      }
      tmp[k++] = p[i];
    }
  };
  rec(0, n);
  return best;
}

// Минимальная покрывающая окружность (Вельцль, ожидаемое O(n))
cr minCircle(ve<pt> p) {
  static mt19937 gen(12345);
  shuffle(all(p), gen);
  cr res(p[0], 0);
  auto out = [&](const pt &q) { return sgn(pt(res.c, q).len() - res.r) > 0; };
  for (size_t i = 1; i < p.size(); i++) {
    if (!out(p[i])) continue;
    res = cr(p[i], 0);
    for (size_t j = 0; j < i; j++) {
      if (!out(p[j])) continue;
      res = cr((p[i] + p[j]) / 2, pt(p[i], p[j]).len() / 2);
      for (size_t k = 0; k < j; k++) {
        if (out(p[k])) res = circum(p[i], p[j], p[k]);
      }
    }
  }
  return res;
}

// Пересечение полуплоскостей. Полуплоскость -- слева от луча p + t * d.
struct HP {
  pt p, d;
  ld ang;
  HP(const pt &_p, const pt &_d) : p(_p), d(_d), ang(atan2l(_d.y, _d.x)) {}
  bool out(const pt &q) const { return sgn(d % (q - p)) < 0; }
      // q строго снаружи
  bool operator<(const HP &o) const { return ang < o.ang; }
};

pt hpInter(const HP &a, const HP &b) {
  return a.p + a.d * (((b.p - a.p) % b.d) / (a.d % b.d));
}
```

```
// Многоугольник пересечения (ccw) или пусто. Добавьте большой
     bounding box, если область не ограничена.
ve<pt> hpi(ve<HP> H) {
  sort(all(H));
  deque<HP> dq;
  for (size_t i = 0; i < H.size(); i++) {
    while (dq.size() > 1 && H[i].out(hpInter(dq[dq.size() -
     1], dq[dq.size() - 2]))) dq.pop_back();
    while (dq.size() > 1 && H[i].out(hpInter(dq[0], dq[1])))
     dq.pop_front();
    if (!dq.empty() && sgn(H[i].d % dq.back().d) == 0) {
      if (sgn(H[i].d * dq.back().d) < 0) return {};
      if (H[i].out(dq.back().p)) dq.pop_back();
      else continue;
    }
    dq.push_back(H[i]);
  }
  while (dq.size() > 2 && dq[0].out(hpInter(dq[dq.size() - 1],
     dq[dq.size() - 2]))) dq.pop_back();
  while (dq.size() > 2 && dq.back().out(hpInter(dq[0], dq[1]))
     ) dq.pop_front();
  if (dq.size() < 3) return {};
  ve<pt> res;
  for (size_t i = 0; i < dq.size(); i++) res.push_back(hpInter
     (dq[i], dq[(i + 1) % dq.size()]));
  return res;
}

// Площадь пересечения круга (c, r) с простым многоугольником
     p (со знаком ориентации), O(n)
ld circlePoly(const ve<pt> &p, const pt &c, ld r) {
  auto arg = [](const pt &a, const pt &b) { return atan2l(a %
     b, a * b); };
  auto tri = [&](const pt &a, const pt &b) -> ld {
    ld r2 = r * r / 2;
    pt d = b - a;
    ld A = (d * a) / d.len2(), B = (a.len2() - r * r) / d.len2
     (), det = A * A - B;
    if (det <= 0) return arg(a, b) * r2;
    ld s = max<ld>(0, -A - sqrtl(det)), t = min<ld>(1, -A +
     sqrtl(det));
    if (t < 0 || 1 <= s) return arg(a, b) * r2;
    pt u = a + d * s, v = a + d * t;
    return arg(a, u) * r2 + (u % v) / 2 + arg(v, b) * r2;
  };
  ld sum = 0;
  for (size_t i = 0; i < p.size(); i++) sum += tri(p[i] - c, p
     [(i + 1) % p.size()] - c);
  return sum;
}
```

WE ARE, MOTHERFUCKERS. WE ARE.

8. Разное

pbds, хеш, быстрый ввод, int128

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

// ordered set: order_of_key(x) -- число элементов < x,
     find_by_order(k) -- k-й (с нуля).
// Для мультимножества: пары (значение, уникальный номер).
using namespace __gnu_pbds;
template <typename T>
using oset = tree<T, null_type, less<T>, rb_tree_tag,
     tree_order_statistics_node_update>;

// Защита от антихеш-тестов: unordered_map<ll, T, SplitMix> (u
     ли gp_hash_table)
struct SplitMix {
  size_t operator()(ull x) const {
    static const ull FIXED = chrono::steady_clock::now().
     time_since_epoch().count();
    x += FIXED + 0x9e3779b97f4a7c15ULL;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
  }
};

// Быстрый ввод целых (fread-буфер)
static char ibuf[1 << 25];
size_t ipos = 0, ilen = 0;
inline int gc() {
  if (ipos == ilen) {
    ilen = fread(ibuf, 1, sizeof(ibuf), stdin);
    ipos = 0;
    if (!ilen) return -1;
  }
  return ibuf[ipos++];
}

inline ll readInt() {
  int c = gc();
  while (c != '-' && (c < '0' || c > '9')) c = gc();
  bool neg = (c == '-');
  if (neg) c = gc();
  ll x = 0;
  for (; c >= '0' && c <= '9'; c = gc()) x = x * 10 + (c - '0'
     );
  return neg ? -x : x;
}

// Вывод __int128
string toString(__int128 x) {
  if (x == 0) return "0";
  bool neg = x < 0;
  if (neg) x = -x;
  string s;
  for (; x; x /= 10) s += char('0' + (int)(x % 10));
  if (neg) s += '-';
  reverse(all(s));
  return s;
}

mt19937_64 rng64(chrono::steady_clock::now().time_since_epoch
     ().count());
```

9. Факты

Обозначения (читать первым)

Факториальные степени. $x^{\underline{k}} = x(x-1)(x-2)\cdots(x-k+1)$ – *убывающая* степень (k множителей, каждый на 1 меньше предыдущего); $x^{\underline{k}} = k!\binom{x}{k}$, это число упорядоченных выборок k элементов из x. $x^{\overline{k}} = x(x+1)\cdots(x+k-1)$ – *возрастающая*. Чёрточка под показателем – убывающая, над ним – возрастающая.

$[P]$ – скобка Айверсона: 1, если утверждение P верно, иначе 0 (например $[n=1]$). $\lfloor x\rfloor$, $\lceil x\rceil$ – пол и потолок, $\{x\} = x - \lfloor x\rfloor$ – дробная часть. $[x^k]F(x)$ – коэффициент при x^k в ряде F. $a \bmod m$ – остаток в $[0, m)$.

Числа. $\binom{n}{k}$ (равно 0 вне $0 \le k \le n$); $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$; F_n – Фибоначчи ($F_0{=}0, F_1{=}1$); C_n – Каталан; D_n – беспорядки (перестановки без неподвижных точек); $S(n,k)$ и $c(n,k)$ – Стирлинга II и I рода; B_n – Белла; $p(n)$ – число разбиений.

Теория чисел. φ – Эйлер, μ – Мёбиус, $d(n)$ (иногда τ) – число делителей, $\sigma(n)$ – их сумма, $v_p(n)$ – показатель простого p в n, $a \perp b$ значит $\gcd(a,b) = 1$. $f * g$ – свёртка Дирихле: $(f*g)(n) = \sum_{d|n} f(d)g(n/d)$; функции $\mathbf{1}(n) = 1$, $\mathrm{id}(n) = n$, $\varepsilon(n) = [n=1]$.

Векторы. $u \cdot v = u_x v_x + u_y v_y$ – скалярное, $u \times v = u_x v_y - u_y v_x$ – векторное (псевдоскалярное): знак > 0, если v повёрнут против часовой от u; $|u \times v|$ – площадь параллелограмма.

Графы. $\deg v$ – степень, α – макс. независимое множество, ν – макс. паросочетание, β – мин. вершинное покрытие, ρ – мин. рёберное покрытие, χ – хроматическое число, $\mathrm{odd}(G)$ – число компонент нечётного размера. $\oplus$ – xor, $\mathrm{mex}(A)$ – наименьшее неотрицательное число не из A. $E[X]$, $V(X)$ – матожидание и дисперсия.

Производящие функции. Обычная (OGF): $\sum a_n x^n$. Экспоненциальная (EGF): $\sum a_n \frac{x^n}{n!}$. EGF удобна для *помеченных* объектов (перестановки, разбиения), OGF – для *непомеченных* (композиции, разбиения чисел).

SUCK IT UP. SUBMIT THE SOLUTION.

Как решать задачи на подсчёт (шпаргалка)

Распознавание. «Сколько способов» → спросите: (1) *порядок важен?* (2) *предметы различимы?* (3) *можно ли повторять?* Выбор k из n без порядка – $\binom{n}{k}$; с порядком – $n^{\underline{k}}$; с повторами без порядка – $\binom{n+k-1}{k}$. Разложение на группы: различимые предметы по различимым ящикам – k^n, по неразличимым – $S(n,k)$; неразличимые предметы по различимым ящикам – шарики и перегородки (ниже).

Приёмы. (a) *Дополнение:* считать «плохие» и вычесть. (b) *Включения-исключения:* условия «ни одно из A_i не выполнено» $= \sum_T (-1)^{|T|} \left|\bigcap_{i\in T} A_i\right|$; удобно, когда пересечение T условий считается формулой (пример: беспорядки, поверхностные отображения, ограничение $x_i \le k$). (c) *Линейность:* матожидание суммы – сумма матожиданий, независимость не нужна. (d) *Биекция:* заменить объект на более простой (ПСП ↔ пути, деревья ↔ коды Прюфера). (e) *Отражение:* пути, касающиеся запрещённой прямой, биективны путям до отражённой конечной точки.

Биномиальные коэффициенты

Базовое. $\binom{n}{k} = \binom{n}{n-k} = \frac{n}{k}\binom{n-1}{k-1}$; Паскаль $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$; $\binom{-n}{k} = (-1)^k\binom{n+k-1}{k}$ (отрицательный верхний индекс сводится к положительному).

Суммы по k. $\sum_k \binom{n}{k} = 2^n$; чётные и нечётные k дают по 2^{n-1}; $\sum_k k\binom{n}{k} = n2^{n-1}$ (выбрать подмножество и в нём «старшего»); $\sum_k \binom{n}{k}\binom{k}{m} = \binom{n}{m}2^{n-m}$ (выбрать m-подмножество, потом любое между ним и целым); $\sum_k \binom{n}{k}^2 = \binom{2n}{n}$; $\sum_k \binom{n-k}{k} = F_{n+1}$ (разбиения n на единицы и двойки). $\sum_{i\le k}(-1)^i\binom{n}{i} = (-1)^k\binom{n-1}{k}$.

Вандермонд: $\sum_k \binom{m}{k}\binom{n}{r-k} = \binom{m+n}{r}$ – выбрать r человек из двух групп. **Клюшка:** $\sum_{i=r}^{n}\binom{i}{r} = \binom{n+1}{r+1}$ (выбор $(r{+}1)$-подмножества по его наибольшему элементу); диагональ $\sum_{i\le k}\binom{n+i}{i} = \binom{n+k+1}{k}$. $\sum_{i=0}^{k}\binom{k}{i}/\binom{n}{i} = \binom{n+1}{n-k+1}/\binom{n}{k}$. $\sum_i \binom{n}{i}i^k = \sum_j S(k,j)\,n^{\underline{j}}2^{n-j}$ (степень i^k раскладываем по убывающим степеням).

Скользящая сумма $B(l,r,n) = \sum_{i=l}^{r}\binom{n}{i}$: $B(l,r,n{+}1) = 2B(l,r,n) + \binom{n}{l-1} - \binom{n}{r}$ – двигать n на единицу за $O(1)$ (приём для задач «сумма биномов по диапазону при растущем n», как в Мо).

Куммер: $v_p\binom{n}{m}$ равно числу переносов при сложении m и $n-m$ в p-ичной системе (показатель простого в биноме). **Лукас:** $\binom{n}{m} \equiv \prod_i \binom{n_i}{m_i} \pmod p$ для простого p, где n_i, m_i – p-ичные цифры. Применять, когда n огромно, а p мал.

Биномы по составному модулю: раскладываем модуль на степени простых, для каждого считаем $\binom{n}{k}$ с учётом v_p (Куммер или Лежандр $v_p(n!) = \sum_i \lfloor n/p^i\rfloor$) и частью без p, затем КТО.

Разложение на слагаемые (шарики и перегородки)

Число решений $x_1 + \cdots + x_n = m$: при $x_i \ge 0$ – $\binom{m+n-1}{n-1}$; при $x_i \ge 1$ – $\binom{m-1}{n-1}$ (вычесть по 1, т.е. «зарезервировать»); при $x_i \ge a_i$ – вычесть a_i и взять первую формулу; при $0 \le x_i \le k$ – включения-исключения: $\sum_{t=0}^{\min(n,\lfloor m/(k+1)\rfloor)}(-1)^t\binom{n}{t}\binom{m-t(k+1)+n-1}{n-1}$ (здесь t – сколько переменных нарушили границу).

n различимых людей по k различимым группам размера $\le m$ (пустые можно): $f(0,k) = 1$, $f(n,k) = k\,f(n{-}1,k) - k\binom{n-1}{m}f(n{-}m{-}1,k{-}1)$.

Пути на сетке. Путей $(0,0) \to (a,b)$ шагами вправо/вверх – $\binom{a+b}{a}$. Пары путей $(0,0) \to (n,m)$ без общих точек, кроме концов (неупорядоченные): $\binom{n+m-2}{n-1}^2 - \binom{n+m-2}{n}\binom{n+m-2}{m}$. Обобщение на несколько путей – Линдстрём–Гессель–Вьенно: det матрицы чисел путей из A_i в B_j.

Задача о голосовании. За A отдано a голосов, за B – $b < a$. Порядков, в которых A *строго* впереди в каждый момент: $\frac{a-b}{a+b}\binom{a+b}{a}$; A *не отстаёт*: $\frac{a-b+1}{a+1}\binom{a+b}{a}$. Это тот же принцип отражения, что и для Каталана.

Каталан и скобочные последовательности

$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{2n+1}\binom{2n+1}{n}$, $C_n = \sum_{i<n} C_i C_{n-1-i}$.

1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900, 2674440.

Что считает: правильные скобочные последовательности (ПСП) длины $2n$; триангуляции выпуклого $(n{+}2)$-угольника; бинарные деревья на n вершинах; пути из $(0,0)$ в (n,n), не поднимающиеся выше диагонали; непересекающиеся хорды $2n$ точек на окружности. Если в задаче видите рекурренту $f_n = \sum f_i f_{n-1-i}$ – это Каталан.

$\sum_{x_0+\cdots+x_k=n}\prod C_{x_i} = \frac{k+1}{n+k+1}\binom{2n+k}{n}$ (лес из $k{+}1$ таких структур).

ПСП: баланс (число «(» минус «)») каждого префикса ≥ 0, итоговый 0. Скобочных слов с n открывающими и $m \le n$ закрывающими, дополняемых до ПСП: $\binom{n+m}{m} - \binom{n+m}{m-1}$. Ровно k пиков «()» (числа Нараяны): $N(n,k) = \frac{1}{n}\binom{n}{k}\binom{n}{k-1}$; столько же плоских корневых деревьев с n рёбрами и k листьями. Скобка с номером p (с нуля) закрывающая в $\sum_{m=0}^{\min(n-1,\lfloor(p-1)/2\rfloor)} C_m C_{n-m-1}$ ПСП. Идея вывода: из $2n{+}1$ циклических сдвигов слова из n «(» и $n{+}1$ «)» ровно один начинается с ПСП.

Числа Стирлинга, Белла, Эйлера

$S(n,k)$ – число разбиений n-элементного множества на k непустых неупорядоченных блоков. $S(n,k) = kS(n{-}1,k) + S(n{-}1,k{-}1)$ (элемент n либо в одном из k блоков, либо отдельно) $= \frac{1}{k!}\sum_j (-1)^{k-j}\binom{k}{j}j^n$ (включения-исключения по пустым блокам; $k!\,S(n,k)$ – число сюръекций). Связь со степенью: $x^n = \sum_k S(n,k)\,x^{\underline{k}}$ – любая степень раскладывается по убывающим; именно так считают $\sum_i i^n$ и $\sum\binom{n}{i}i^n$. Ряд: $S(n,k) = [x^{n-k}]\prod_{i=1}^{k}\frac{1}{1-ix}$ (фиксированное k); вся строка n одной свёрткой: $S(n,k) = [x^k]\left(\sum_i \frac{i^n}{i!}x^i\right)\left(\sum_j \frac{(-1)^j}{j!}x^j\right)$ (NTT, см. `opts/poly.cpp`).

$c(n,k)$ – число перестановок n элементов с ровно k циклами (беззнаковые числа I рода): $c(n,k) = c(n{-}1,k{-}1) + (n{-}1)c(n{-}1,k)$. Многочлены: $x^{\overline{n}} = \sum_k c(n,k)x^k$ и $x^{\underline{n}} = \sum_k (-1)^{n-k}c(n,k)x^k$. Вся строка n – коэффициенты $\prod_{t<n}(x+t)$, считаются разделяй-и-властвуй с FFT за $O(n\log^2 n)$. Столбец k: $\frac{(-\ln(1-x))^k}{k!}$.

Обращение Стирлинга: $P(k)=\sum_i S(k,i)Q(i) \iff Q(k)=\sum_i(-1)^{k-i}c(k,i)P(i)$ – если нашли сумму, выраженную через степени, обратно восстановим коэффициенты по убывающим степеням.

Блоки, где любые два элемента одного блока отличаются не менее чем на d: $S(n-d+1,\,k-d+1)$ при $k\ge d$.

Белла $B_n=\sum_k S(n,k)=\sum_k \binom{n-1}{k-1}B_{n-k}$ – число всех разбиений; 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975, 678570, 4213597.

Эйлеровы $E(n,k)$ – перестановки с ровно k подъёмами ($p_i<p_{i+1}$): $E(n,k)=(n-k)E(n-1,k-1)+(k+1)E(n-1,k)=\sum_j(-1)^j\binom{n+1}{j}(k+1-j)^n$; $\sum_k E(n,k)=n!$.

Перестановки и симметрии

Беспорядки (без неподвижных точек): $D_n=(n-1)(D_{n-1}+D_{n-2})=nD_{n-1}+(-1)^n=n!\sum_{t\le n}\frac{(-1)^t}{t!}=\lfloor n!/e\rceil$ $(n\ge 1)$; $\sum_k\binom{n}{k}D_{n-k}=n!$ (перестановку можно задать числом k неподвижных и беспорядком остальных); 1, 0, 1, 2, 9, 44, 265, 1854, 14833, 133496. Типичное применение: «сколько перестановок, где ровно k неподвижных» $=\binom{n}{k}D_{n-k}$.

Средние для случайной перестановки: циклов – H_n, рекордов – H_n, неподвижных точек – 1, инверсий – $\frac{n(n-1)}{4}$.

Перестановки с циклами только длин из множества T: $\sum_n g_T(n)\frac{x^n}{n!}=\exp\left(\sum_{n\in T}\frac{x^n}{n}\right)$ (EGF).

Бернсайд: число классов эквивалентности $=\frac{1}{|G|}\sum_{g\in G}|X^g|$, где X^g – объекты, не меняющиеся при g. Ожерелья из n бусин, k цветов, с точностью до поворота: $N=\frac{1}{n}\sum_{d|n}\varphi(d)k^{n/d}$. Браслеты (ещё и отражения): $\frac{1}{2}\left(N+k^{(n+1)/2}\right)$ при нечётном n и $\frac{1}{2}\left(N+\frac{(k+1)k^{n/2}}{2}\right)$ при чётном.

Бинарных строк ровно с m различными подпоследовательностями (считая пустую): $\varphi(m+1)$.

Разбиения числа

$p(n)$ – число способов записать n суммой натуральных без учёта порядка. Пентагональная теорема: $\prod_{n\ge1}(1-x^n)=\sum_{k\in\mathbb{Z}}(-1)^k x^{k(3k-1)/2}$; отсюда $p(n)=\sum_{k\ne0}(-1)^{k+1}p\left(n-\frac{k(3k-1)}{2}\right)$ за $O(n\sqrt{n})$ (в сумме только $O(\sqrt{n})$ слагаемых). $p(n)\approx\frac{1}{4\sqrt{3}n}e^{\pi\sqrt{2n/3}}\approx\frac{0.145}{n}e^{2.56\sqrt{n}}$.

$p(0..10)=1,1,2,3,5,7,11,15,22,30,42$; $p(20)=627$, $p(50)=204226$, $p(100)=190569292$.

Деревья, остовы, эйлеровы циклы

Кэли: помеченных деревьев на n вершинах n^{n-2}; код Прюфера – биекция с последовательностями из $n-2$ чисел (повторяя «удали наименьший лист и запиши его соседа»). Деревьев с заданными степенями d_v ($\sum d_v=2n-2$, $d_v\ge1$): $\frac{(n-2)!}{\prod(d_v-1)!}$ (вершина v встречается в коде d_v-1 раз). Лесов из m деревьев на n вершинах, где выделенные вершины $1..m$ лежат в разных деревьях: $m\,n^{n-m-1}$ †. $\tau(K_{a,b})=a^{b-1}b^{a-1}$.

Кирхгоф (число остовов): построить матрицу L: для ребра $u\to v$ сделать $L[u][u]-=1$, $L[v][v]+=1$ (неориентированное ребро – оба направления). Минор без строки и столбца r – число остовных деревьев, направленных от r (для неориентированного – обычных остовов, любой r). С весами рёбер – сумма по остовам произведений весов. Чтобы получить деревья, направленные к корню, ставьте $L[u][u]+=1$ вместо $L[v][v]$.

BEST: число эйлеровых циклов орграфа (как циклических последовательностей рёбер) $=t_w(G)\prod_v(\deg_{out}(v)-1)!$, где t_w – число остовов, направленных к w †.

Эйлеров путь: в неориентированном – связность по рёбрам и ≤2 нечётных вершин; в орграфе – $\text{out}-\text{in}\in\{0,\pm1\}$, не более одной «начальной» $(+1)$ и одной «конечной» (-1).

Простые факты о деревьях: $\sum_v(\deg v-2)=-2$; листьев не меньше двух; диаметр находится двумя обходами (самая дальняя от любой вершины a, затем самая дальняя от неё); центр – одна или две средние вершины диаметра, радиус $\lceil\text{diam}/2\rceil$.

Производящие функции (помеченные объекты, EGF)

Пара независимых частей – произведение $A\cdot B$; последовательность – $\text{Seq}(A)=\frac{1}{1-A}$; множество – $\text{Set}(A)=e^A$; цикл – $\text{Cycle}(A)=\ln\frac{1}{1-A}$. Следствия: $S(n,k)\leftrightarrow\frac{(e^x-1)^k}{k!}$, $c(n,k)\leftrightarrow\frac{(-\ln(1-x))^k}{k!}$, $D_n\leftrightarrow\frac{e^{-x}}{1-x}$, Белл $\leftrightarrow e^{e^x-1}$. Непомеченные объекты: $\text{MSet}(A)=\exp\left(\sum_{k\ge1}\frac{A(z^k)}{k}\right)$ (разбиения чисел при $A=\frac{z}{1-z}$), $\text{Cycle}(A)=\sum_{n\ge1}\frac{\varphi(n)}{n}\ln\frac{1}{1-A(z^n)}$ (ожерелья).

Суммы, ряды, рекуррентности

$\sum_{i\le n}i^2=\frac{n(n+1)(2n+1)}{6}$, $\sum i^3=\frac{n^2(n+1)^2}{4}$, $\sum i^4=\frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$; $\sum_{i=1}^n ia^i=\frac{a(na^{n+1}-(n+1)a^n+1)}{(a-1)^2}$.

$e^x=\sum\frac{x^k}{k!}$, $\ln(1+x)=\sum_{k\ge1}\frac{(-1)^{k+1}x^k}{k}$, $(1-x)^{-n}=\sum_k\binom{n+k-1}{n-1}x^k$, $\arctan x=\sum(-1)^k\frac{x^{2k+1}}{2k+1}$.

Линейная рекуррента $a_n=\sum_{i\le k}c_ia_{n-i}$: решение – $a_n=\sum d_jr_j^n$ по корням r_j характеристического многочлена $x^k-\sum c_ix^{k-i}$ (кратный корень даёт множитель-многочлен). Практически: a_n за $O(k^2\log n)$ возведением x в степень по модулю характеристического многочлена; по $2k$ первым членам рекурренту восстанавливает Берлекэмп–Мэсси (`math/linrec.cpp`).

STAY HARD.

Модульная арифметика

Эйлер, Ферма, обобщённая Эйлера: $a^{\varphi(m)}\equiv1\pmod{\gcd(a,m)=1}$; для любых a и $x\ge\log_2 m$: $a^x\equiv a^{\varphi(m)+(x\bmod\varphi(m))}\pmod m$ (башни степеней). Вильсон: $(p-1)!\equiv-1\pmod p$; отсюда $n!\,(p-1-n)!\equiv(-1)^{n+1}\pmod p$ – если n близко к p (например $p-1-n$ мало), $n!\bmod p$ считается через маленький факториал.

Как пользоваться. Деление по модулю – умножение на обратный ($a^{-1}=a^{p-2}$ для простого p, либо расширенный Евклид для любого m при $\gcd(a,m)=1$). Если $\gcd(a,m)>1$, делить нельзя: хранить числа как пару (степени простых, остаток) или работать с $\mathbb{Z}$ по КТО. Башня степеней $a^{b^c}\bmod m$ – рекурсивно по $\varphi(m)$ (формула выше).

Первообразные корни есть при $m=1,2,4,p^k,2p^k$ (p нечётное); их ровно $\varphi(\varphi(m))$. Порядок элемента делит $\varphi(m)$; g – корень $\iff g^{(p-1)/q}\ne1$ для всех простых $q\mid p-1$.

Квадратичные вычеты: $\left(\frac{a}{p}\right)\equiv a^{(p-1)/2}$; взаимность $\left(\frac{p}{q}\right)\left(\frac{q}{p}\right)=(-1)^{\frac{p-1}{2}\cdot\frac{q-1}{2}}$ (нечётные простые $p\ne q$). Для нечётного p и $a\not\equiv0$ корней $x^2=a$ ровно 0 или 2.

φ: $\sum_{d|n}\varphi(d)=n$; $\varphi(mn)=\varphi(m)\varphi(n)\frac{g}{\varphi(g)}$, $g=\gcd(m,n)$; $\sum_{k\le n,\gcd(k,n)=1}k=\frac{n\varphi(n)}{2}$ $(n>1)$.

Диофант $ax+by=c$: решения есть $\iff\gcd(a,b)\mid c$; все: $(x_0+k\frac{b}{g},\ y_0-k\frac{a}{g})$. При $\gcd(a,b)=1$ число решений в $x,y\ge0$: $\frac{n}{ab}-\{\frac{b'n}{a}\}-\{\frac{a'n}{b}\}+1$, где $aa'\equiv1\pmod b$, $bb'\equiv1\pmod a$.

Степени и НОД: $\gcd(a^n-1,a^m-1)=a^{\gcd(n,m)}-1$; $v_p(n!)=\sum_{i\ge1}\lfloor n/p^i\rfloor=\frac{n-s_p(n)}{p-1}$ (s_p – сумма p-ичных цифр; число нулей в конце $n!$ – это v_5).

НОД/НОК: $\gcd(a,\text{lcm}(b,c))=\text{lcm}(\gcd(a,b),\gcd(a,c))$ и двойственное; $\gcd(\text{lcm}(a,b),\text{lcm}(b,c),\text{lcm}(a,c))=\text{lcm}(\gcd(a,b),\gcd(b,c),\gcd(a,c))$.

Мёбиус и суммы по делителям

$\mu(n)=0$ при квадрате в n, иначе $(-1)^{\#\text{простых}}$; $\sum_{d|n}\mu(d)=[n=1]$.
$g(n)=\sum_{d|n}f(d)\iff f(n)=\sum_{d|n}\mu(d)g(\frac{n}{d})$; $g(n)=\sum_{n|d}f(d)\iff f(n)=\sum_{n|d}\mu(\frac{d}{n})g(d)$;
$g(n)=\sum_{m\le n}f(\lfloor\frac{n}{m}\rfloor)\iff f(n)=\sum_{m\le n}\mu(m)g(\lfloor\frac{n}{m}\rfloor)$.

Число делителей и их сумма: $d(n)=\prod(e_i+1)$, $\sigma(n)=\prod\frac{p_i^{e_i+1}-1}{p_i-1}$ по разложению $n=\prod p_i^{e_i}$; суммы по всем $k\le n$ считаются перестановкой суммирования: $\sum_{k\le n}d(k)=\sum_{k\le n}\lfloor\frac{n}{k}\rfloor$, $\sum_{k\le n}\sigma(k)=\sum_{k\le n}k\lfloor\frac{n}{k}\rfloor$.

Взаимно простые пары. $\#\{(i,j)\le n : \gcd(i,j)=1\} = \sum_{d\le n}\mu(d)\lfloor\frac{n}{d}\rfloor^2 = 2\sum_{i\le n}\varphi(i)-1$; пар с $\gcd=k$ – то же для $\lfloor n/k\rfloor$. Это основной приём для «посчитать пары с условием на НОД»: перейти к μ и сгруппировать по d.

Блоки $\lfloor n/i\rfloor$. Значение $\lfloor n/l\rfloor$ постоянно на $[l,\ r]$, $r=\lfloor n/\lfloor n/l\rfloor\rfloor$: цикл `for(l=1;l<=n;l=r+1){r=n/(n/l);...}` делает $O(\sqrt{n})$ шагов. Применять к любым суммам вида $\sum_i f(\lfloor n/i\rfloor)\,g(i)$ с префиксными суммами g.

Свёртки: $\sum_{d|n}\varphi(d)=n$, $\sum_{k\le n}\gcd(k,n)=\sum_{d|n}d\varphi(\frac{n}{d})$, $\sum_{k\le n}\operatorname{lcm}(k,n)=\frac{n}{2}\big(\sum_{d|n}d\varphi(d)+1\big)$, $\sum_{i,j\le n}\operatorname{lcm}(i,j)=\sum_{l\le n}S\big(\lfloor\frac{n}{l}\rfloor\big)^2\sum_{d|l}\mu(d)\,l\,d$, где $S(m)=\frac{m(m+1)}{2}$.
Бесквадратных $\le N$: $\sum_{k\le\sqrt{N}}\mu(k)\lfloor N/k^2\rfloor$. Значений $\lfloor n/i\rfloor$ не более $2\sqrt{n}$ (блоки одинаковых значений); $\sum_{k\le n}\lfloor n/k\rfloor\approx n\ln n$; $\sum_{d|n}d=O(n\log\log n)$.

Идея: суммы по делителям и свёртки. Хотим $g(m)=\sum_{d|m}f(d)$ для всех $m\le n$ – «префиксные суммы» по решётке делимости. Делаем по одному простому p: для $i=1..n/p$ по возрастанию $f[ip]\mathrel{+}=f[i]$ (после всех простых получается g), всего $O(n\log\log n)$. Обратное – *преобразование Мёбиуса*: те же простые, но i по убыванию и $f[ip]\mathrel{-}=f[i]$ (это и есть $f=\mu*g$). Для сумм по кратным ($g(d)=\sum_{d|m}f(m)$) циклы идут в обратную сторону. Свёртки: **НОД**: $c_k=\sum_{\gcd(i,j)=k}a_ib_j$ – сумма по кратным → почленное произведение → Мёбиус по кратным. **НОК**: то же по делителям. Свёртка Дирихле $\sum_{ij=n}a_ib_j$ – за $O(n\log n)$ прямым перебором; $\mu*\mathbf{1}=[n{=}1]$, $\varphi*\mathbf{1}=\mathrm{id}$. Код: `math/dirichlet.cpp`.

Суммы квадратов, тройки, решётка

Тройки: $a=k(m^2{-}n^2)$, $b=2kmn$, $c=k(m^2{+}n^2)$, $m>n>0$, $\gcd(m,n)=1$, $m\not\equiv n\ (2)$ – все примитивные (при $k{=}1$). Пар (a,b), $a,b>0$, с $a^2{+}b^2=n^2$: $\prod_{p\equiv1\,(4)}(2\alpha_p{+}1)-1$.

n – сумма двух квадратов $\iff$ простые $\equiv 3\ (4)$ входят в чётных степенях; $r_2(n)=4(d_1(n){-}d_3(n))$ (делители $\equiv 1,3 \bmod 4$); любое n – сумма четырёх квадратов, $r_4(n)=8\sum_{d|n,\,4\nmid d}d$; $r_8(n)=16\sum_{d|n}(-1)^{n+d}d^3$; сумма трёх квадратов невозможна ровно для $n=4^a(8b{+}7)$.

Пик: $S=I+\frac{B}{2}-1$; целых точек на отрезке $\gcd(|dx|,|dy|){+}1$.

Фибоначчи

$F_{2n}=F_n(2F_{n+1}{-}F_n)$, $F_{2n+1}=F_n^2+F_{n+1}^2$; $\gcd(F_a,F_b)=F_{\gcd(a,b)}$; Кассини $F_{n-1}F_{n+1}-F_n^2=(-1)^n$; $\sum_{i\le n}F_i=F_{n+2}-1$, $\sum F_i^2=F_nF_{n+1}$. n – число Фибоначчи $\iff$ $5n^2\pm4$ – точный квадрат. Цекендорф: единственное разложение без соседних слагаемых. Период Пизано $\pi(m)\le 6m$, равенство при $m=2\cdot5^k$, $k\ge1$.

Асимптотики

$n!\sim\sqrt{2\pi n}\big(\frac{n}{e}\big)^n\big(1+\frac{1}{12n}+\frac{1}{288n^2}-\frac{139}{51840n^3}\big)$; $\ln n!=n\ln n-n+\frac12\ln(2\pi n)+\frac{1}{12n}-\frac{1}{360n^3}+\frac{1}{1260n^5}$; $\binom{2n}{n}\approx\frac{4^n}{\sqrt{\pi n}}$; $H_n\approx\ln n+0.5772157+\frac{1}{2n}$; $\pi(x)\approx x/\ln x$.

Таблицы

k	1	2	3	4	5	6	7	8	9	10
$\pi(10^k)$	4	25	168	1229	9592	78498	664579	5761455	50847534	455052511

Наибольшее число делителей ($N<$): $5\cdot10^4$ – 100 (45360); 10^7 – 448 (8648640); 10^{10} – 2304 (6983776800) †; 10^{19} – 161280.

$N\le$	10^3	10^6	10^9	10^{12}	10^{15}	10^{18}
n	840	720720	735134400	963761198400	866421317361600	897612484786617600
$d(n)$	32	240	1344	6720	26880	103680

Пределы типов: $12!<2^{31}\le13!$; $20!<2^{63}\le21!$; $170!<\mathrm{DBL_MAX}<171!$; $F_{46}<2^{31}$, $F_{92}<2^{63}$; $C_{19}<2^{31}$, $C_{35}<2^{63}$ (Каталан); $\binom{66}{33}<2^{63}$.

Простые. NTT: $998244353=119\cdot2^{23}{+}1$, $167772161=5\cdot2^{25}{+}1$, $469762049=7\cdot2^{26}{+}1$ (корень 3), $962592769=459\cdot2^{21}{+}1$; 64-бит: $29\cdot2^{57}{+}1$ ($g{=}3$), $27\cdot2^{56}{+}1$ ($g{=}5$), $237\cdot2^{52}{+}1$ ($g{=}5$). $10^9{+}\{7,9,21,33,87,93,97,103,123\}$; $10^{18}{+}\{3,9,31,79,177,183,201,283\}$. Для хеша: 970592641 (30 бит)†, 31443539979727 (45), 3006703054056749 (52), 1791791791, 1234676767, 1000575757, 998696969, 996179179, 991676767.

Миллер–Рабин, детерминированные базы: $n<4759123141$: $\{2,7,61\}$; $n<2152302898747$: $\{2,3,5,7,11\}$; $n<2^{64}$: $\{2,325,9375,28178,450775,9780504,1795265022\}$ или первые 12 простых.

WHO'S GONNA CARRY THE BOATS AND THE LOGS?

Геометрия на плоскости

Базовые операции. Знак $u\times v$: >0 – поворот против часовой, <0 – по часовой, $=0$ – коллинеарны; на этом строятся все предикаты (выпуклость, пересечение, точка в многоугольнике). Расстояние от точки p до прямой через a,b: $\frac{|(b-a)\times(p-a)|}{|b-a|}$; проекция $a+t(b-a)$, $t=\frac{(p-a)\cdot(b-a)}{|b-a|^2}$; отражение точки от прямой: $2\cdot\mathrm{proj}-p$. Пересечение прямых $a+td$ и $c+se$: $t=\frac{(c-a)\times e}{d\times e}$ (при $d\times e=0$ – параллельны). *Хранить целые координаты в ll* и сравнивать точно; `double` – только там, где иначе нельзя (окружности, углы). **Треугольник** $(a,b,c,\ p=\frac{a+b+c}{2})$: $S=\sqrt{p(p-a)(p-b)(p-c)}=\frac12|\vec{AB}\times\vec{AC}|$; $R=\frac{abc}{4S}$, $r=\frac{S}{p}$; медиана $m_a=\frac12\sqrt{2b^2+2c^2-a^2}$; биссектриса $\ell_a=\sqrt{bc\,(1-(\frac{a}{b+c})^2)}$ (делит BC как $c{:}b$); теоремы синусов ($\frac{a}{\sin\alpha}=2R$) и косинусов. Инцентр $=\frac{aA+bB+cC}{a+b+c}$. O – центр описанной окружности: $H=A{+}B{+}C-2O$, центроид $G=\frac{2O+H}{3}$ (прямая Эйлера).

Четырёхугольник (стороны a,b,c,d, диагонали e,f, угол между диагоналями θ, $F=b^2{+}d^2{-}a^2{-}c^2$): $4S=2ef\sin\theta=F\tan\theta=\sqrt{4e^2f^2-F^2}$. Вписанный: $ef=ac+bd$ (Птолемей), $S=\sqrt{(p-a)(p-b)(p-c)(p-d)}$.

Многоугольник: $2S=\sum p_i\times p_{i+1}$ (>0 против часовой); центр масс $\frac{1}{6S_\pm}\sum(p_i{+}p_{i+1})(p_i\times p_{i+1})$ при $S_\pm=\frac12\sum p_i\times p_{i+1}$.

Окружности: расстояние между центрами d; хорда пересечения на расстоянии $a=\frac{r_1^2-r_2^2+d^2}{2d}$ от центра 1, полудлина $h=\sqrt{r_1^2-a^2}$; радикальная ось: $2(c_2{-}c_1)\cdot X=|c_2|^2-|c_1|^2-r_2^2+r_1^2$. Касательные из p: пересечь окружность с окружностью $(p,\sqrt{|pc|^2-r^2})$ (длина касательной $\sqrt{|pc|^2-r^2}$). Площадь *сектора* $\frac{r^2\theta}{2}$, *сегмента* (хорда с центральным углом θ) $\frac{r^2}{2}(\theta-\sin\theta)$; площадь *линзы* двух окружностей расстояния d: $r_1^2\arccos\frac{d^2+r_1^2-r_2^2}{2dr_1}+r_2^2\arccos\frac{d^2+r_2^2-r_1^2}{2dr_2}-\frac12\sqrt{(-d+r_1+r_2)(d+r_1-r_2)(d-r_1+r_2)(d+r_1+r_2)}$.

Углы: угол между векторами лучше брать как $\mathrm{atan2}(u\times v,\ u\cdot v)$ (не arccos). Поворот на φ: $(x\cos\varphi-y\sin\varphi,\ x\sin\varphi+y\cos\varphi)$. Сферические: $x=r\sin\theta\cos\phi$, $y=r\sin\theta\sin\phi$, $z=r\cos\theta$.

3D: объём тетраэдра $\frac16|(b-a)\cdot((c-a)\times(d-a))|$. Эйлер для выпуклого многогранника и связного плоского графа: $V-E+F=2$; планарный граф: $E\le3V-6$ (двудольный: $E\le2V-4$).

Среднее и центр. Среднее арифметическое минимизирует $\sum|x-p_i|^2$: $f(q)=f(\bar p)+n|q-\bar p|^2$ (для суммы расстояний – медиана). Триангуляция выпуклого n-угольника: двойственный граф – дерево из $n{-}2$ вершин со степенями ≤3; триангуляций C_{n-2}.

Делоне: вне описанной окружности каждого треугольника нет других точек; единственна, если нет 4 точек на одной окружности; максимизирует минимальный угол. Вороной – пересечение полуплоскостей серединных перпендикуляров к соседям по Делоне.

Теоремы о графах

Паросочетания. Кёниг (двудольный): макс. паросочетание = мин. вершинное покрытие. Покрытие по Кёнигу: DFS из свободных вершин левой доли по неприкрытым и по паросочетанию обратно; $L^-\cup R^+$ – покрытие, $L^+\cup R^-$ – независимое множество. Галлаи: $\alpha+\beta=n$, $\nu+\rho=n$ (без изолированных). Холл: паросочетание, насыщающее L, $\iff$ $|N(S)|\ge|S|\ \forall S\subseteq L$. Татт–Берж: $\nu(G)=\min_U\frac12(|V|+|U|-\mathrm{odd}(G-U))$.

Дилворт: макс. антицепь = мин. покрытие цепями = $n-$ макс. паросочетание в двудольном графе строгого порядка (нужно *транзитивное замыкание*). Антицепь: берём вершины, ни одна из двух копий которых не в покрытии Кёнига. Мин. покрытие DAG непересекающимися путями = $n-$ паросочетание (без замыкания).

Степенные последовательности. Эрдёш–Галлаи: $d_1 \geq \cdots \geq d_n$ – степени простого графа $\iff \sum d_i$ чётна и $\forall k: \sum_{i\leq k} d_i \leq k(k-1) + \sum_{i>k} \min(d_i, k)$. Хавел–Хакими: убрать наибольшее d_1, вычесть 1 у следующих d_1 больших.

Раскраска. Хроматический многочлен: путь/дерево $t(t-1)^{n-1}$, цикл $(t-1)^n + (-1)^n(t-1)$, клика $t(t-1)\cdots(t-n+1)$; $P(G) = P(G-e) - P(G/e)$. Брукс: $\chi \leq \Delta$ (кроме клик и нечётных циклов), Визинг: рёберная $\leq \Delta+1$ (двудольный – Δ). Мантель: без треугольников $\leq \lfloor n^2/4 \rfloor$ рёбер.

Потоки. Макс-флоу = мин-разрез. Поток с нижними границами: ребро $u\to v$ с $[l, c]$ заменяем ребром ёмкости $c-l$ и сдвигаем баланс ($\mathrm{ex}[v]+=l$, $\mathrm{ex}[u]-=l$); добавляем $S\to v$ ёмкости $\mathrm{ex}[v]$ при $\mathrm{ex}[v] > 0$ и $v\to T$ при < 0; допустимая циркуляция есть $\iff$ макс-флоу насыщает все рёбра из S. Менгер: число рёберно-непересекающихся s–t путей = мин. разрез. Гомори–Ху: $n-1$ потоков строят дерево всех мин. разрезов. Стабильные паросочетания: заявки в порядке предпочтений, отвергнутые идут дальше.

Матроиды. Пересечение двух матроидов: граф замен $A \setminus u \cup v \in I_1$ (рёбра $u\to v$), $\in I_2$ ($v\to u$); источники $\{z : A\cup z \in I_1\}$, стоки $\{z : A\cup z \in I_2\}$; кратчайший путь $S\to T$ увеличивает A (взвешенный – кратчайший по весу, среди них по числу рёбер).

Регулярный двудольный граф ($\deg = k$ у всех): всегда есть совершенное паросочетание, и граф раскладывается на k совершенных паросочетаний (рёберная раскраска за k цветов).

Подсчёт путей: число путей длины k из s в t – $(A^k)_{st}$ (возведение матрицы смежности в степень), замкнутых путей – $\mathrm{tr}A^k$.

Цепи Маркова на графе. Случайное блуждание по связному неориентированному неразобщённому графу: стационарное $\pi_v \propto \deg v$. Поглощающая цепь: $N = (I-Q)^{-1}$, ожидаемое время $= N\mathbf{1}$, вероятности поглощения $= NR$.

THEY DON'T KNOW US, SON.

Игры

Как решать. (1) Свести игру к *независимым* подыграм (куча, ряд, компонента): ходы в одной не влияют на другую. (2) Для каждой подигры посчитать g динамикой (перебор ходов, mex по значениям Гранди результатов). (3) Взять хог g всех подыгр: ненулевой – выигрывает ходящий. Если игра на графе с циклами, решаем ретроанализом (позиция проигрышна, если все ходы ведут в выигрышные).

Шпрага–Гранди: $g(v) = \mathrm{mex}\{g(u) : v \to u\}$; сумма игр – хог значений; позиция проигрышна $\iff g = 0$.

Ним: ходит первый и выигрывает $\iff \bigoplus a_i \neq 0$. Мизер-ним (проигрывает взявший последним): как ним, если есть куча > 1; иначе первый выигрывает при *чётном* числе единиц. Лестничный ним: считаются только нечётные ступени. Бэш (берём $1..k$): $g = n \bmod (k+1)$. Уайтхоф (две кучи, ходы: из одной или поровну из обеих): P-позиции $(\lfloor k\varphi \rfloor, \lfloor k\varphi^2 \rfloor)$, $\varphi = \frac{1+\sqrt{5}}{2}$. Кайлс: $g(0..12) = 0, 1, 2, 3, 1, 4, 3, 2, 1, 4, 2, 6, 4$ (далее период 12 начиная с $n = 71$).

Игры на графах без циклов – динамика по порядку; с циклами – ретроанализ от терминальных позиций (счётчики выходов). Минимакс: альфа-бета, порядок ходов по оценке.

Вероятность

Как считать. (1) Матожидание суммы – сумма матожиданий: разбить на индикаторы [событие$_i$], каждый посчитать отдельно. Например, E[число различных значений] $= N\left(1 - (1 - \frac{1}{N})^n\right)$ при n равномерных выборках из N значений, $E[\min] = \sum_k P(\min > k)$. (2) Условное: полная вероятность по первому шагу, система линейных уравнений (Гаусс) или $(I - Q)^{-1}$. (3) Блуждание на отрезке $[0, L]$ с поглощением на концах: из i вероятность дойти до L равна i/L, ожидаемое время $i(L - i)$.

$E[X] = \sum_{k\geq 1} P(X \geq k)$; линейность; $V(aX+bY) = a^2V(X)+b^2V(Y)$ при независимости. Бином $\mu = np$, $\sigma^2 = np(1-p)$; геометрическое (до первого успеха) $\mu = \frac{1}{p}$, $\sigma^2 = \frac{1-p}{p^2}$; Пуассон $\mu = \sigma^2 = \lambda$ ($\mathrm{Bin}(n, p) \approx \mathrm{Po}(np)$); равномерное $[a, b]$: $\mu = \frac{a+b}{2}$, $\sigma^2 = \frac{(b-a)^2}{12}$; нормальное $f = \frac{1}{\sqrt{2\pi}\sigma} e^{-(x-\mu)^2/2\sigma^2}$.

Собиратель купонов: nH_n. Дни рождения: коллизия с вер. $\geq \frac{1}{2}$ при $n \approx \sqrt{2N \ln 2}$ (23 для N=365). Быстрая сортировка: $2(n+1)H_n - 4n$ сравнений. Максимум n равномерных на $[0, 1]$: $\frac{n}{n+1}$.

Ожидание ожидания паттерна в потоке честной монеты: $\sum_{k:\ \text{префикс длины } k=\text{суффикс}} 2^k$ (HH – 6, HT – 4, HHH – 14).

Приёмы и грабли

Сложность за 1 с $\approx 10^8$ простых операций: $n \leq 10$ – $n!$; ≤ 20 – $2^n n$; ≤ 40 – meet-in-the-middle; ≤ 100 – n^4; ≤ 500 – n^3; ≤ 5000 – n^2; $\leq 10^5$ – $n \log^2 n$; $\leq 10^6$ – $n \log n$.

Мо: блок $n/\sqrt{q}$, $O(n\sqrt{q})$; с обновлениями блок $n^{2/3}$, сортировка по $(\lfloor t/K \rfloor, \lfloor l/K \rfloor, r)$, $O(n^{5/3})$.

Оптимизации ДП. Неравенство четырёхугольника для w: $w(a, c) + w(b, d) \leq w(a, d) + w(b, c)$ ($a \leq b \leq c \leq d$) $\Rightarrow$ монотонность опта: D&C ($O(n \log n)$ на слой), Кнут для интервалов $opt[l][r-1] \leq opt[l][r] \leq opt[l+1][r]$ ($O(n^2)$; нужна ещё монотонность w по вложению). Выпуклость по числу групп $\Rightarrow$ лямбда-оптимизация (бинпоиск штрафа).

Код Грея: $g(i) = i \oplus (i \gg 1)$, соседние отличаются в бите. **Битовые:** `x&-x` – младший бит; `__builtin_ctz/clz/popcount`; перебор подмасок `for(s=m;s;s=(s-1)&m)`; битсет ускоряет в 64 раза.

Грабли. Деление отрицательных округляет к нулю: $\lfloor a/b \rfloor$ при $a < 0$ писать вручную; `a%b` может быть отрицательным; `abs(INT_MIN)`; `int*int` переполняется до приведения; `1<<31` – UB, нужно `1LL<<k`; `sqrt` от `long long` теряет точность – править ± 1; в компараторе строгий порядок (иначе UB); не хранить `eps`-сравнения в `std::set` без нужды; `unordered_map` взламывается – свой хеш; рекурсия $> 10^5$ глубины – `ulimit -s unlimited`; `long double` 64-битная мантисса – точные целые до 2^{63}.

Отладка. Минимизировать тест генератором; сравнить с брутфорсом; `-fsanitize=address,undefined -D_GLIBCXX_DEBUG`; проверить границы n=1, пустые, все равные, максимальный тест на время и память; на WA – печатать промежуточные значения на малом тесте руками.

STAY FOCUSED. STAY FUCKING HARD.

Ловкие приёмы и когда их применять

Бинпоиск по ответу: если «подходит x» монотонно по x – искать границу; проверку писать жадно или потоком. **Параллельный бинпоиск:** много запросов «первый момент, когда выполнено» – бинпоиск каждого, но проверки для всех запросов делаются одним проходом по моментам.

Оффлайн: если запросы не зависят от ответов – отсортировать и обработать в удобном порядке (по правой границе + Фенвик; Мо; дерево отрезков по времени + откат СНМ для динамической связности). **Разделяй и властвуй:** посчитать вклад пар через середину, рекурсивно левую и правую половины; на деревьях – центроиды.

Meet-in-the-middle: $n \leq 40$ – разбить на две половины по 2^{20}, посчитать множества и склеить сортировкой или двумя указателями. **Битсет:** перебор вида «есть ли сумма s» ускоряется в 64 раза сдвигами `bitset`.

Корневая декомпозиция: блок размера $\sqrt{n}$: обновление за $O(\sqrt{n})$, запрос за $O(1)$ или наоборот; «тяжёлые/лёгкие» по степени: для тяжёлых ($> \sqrt{m}$) таблицы, для лёгких – перебор.

Хеширование: модуль $2^{61} - 1$ со случайной базой (не 2^{64} – ломается Туэ–Морса); для деревьев – хеш поддерева как функция от отсортированных хешей детей. **Рандом:** равновероятный выбор вместо перебора, отжиг/hill climbing для оптимизационных задач без структуры.

Деление с округлением. В C++ `a/b` округляет к нулю. $\lfloor a/b \rfloor$ при $a < 0$: `a/b - (a%b!=0 && ((a%b<0)!=(b<0)))`; $\lceil a/b \rceil = \lfloor (a + b - 1)/b \rfloor$ при $a \geq 0, b > 0$. Остаток в $[0, m)$: `((a%m)+m)%m`.